Grado en Ingeniería Informática

Tecnologías de Desarrollo de Software

Patrones de Diseño

(Parte 3. Patrones de Comportamiento)

Facultad de Informática

UNIVERSIDAD DE MURCIA

Departamento Informática y Sistemas

Contenidos

- Introducción a los patrones de diseño
- Patrones de **creación**
  - Factoría Abstracta, *Builder*, Método Factoría, Singleton
- Patrones **estructurales**
  - Adaptador, Composite, Decorador, Fachada
- Patrones de **comportamiento**
  - Command, Iterador, Observer, Estado, Estrategia, Método Plantilla

Patrones de Comportamiento

- Soluciones que enfatizan la colaboración entre objetos.
- Basados en composición excepto *Template Method* que *se* basa en la herencia.

Patrón *Método Plantilla*

Propósito

Definir operaciones como esquemas de algoritmos que difieren algunos pasos a operaciones implementadas en las subclases. Permite a las subclases redefinir ciertos pasos de un algoritmo sin cambiar la estructura del algoritmo.

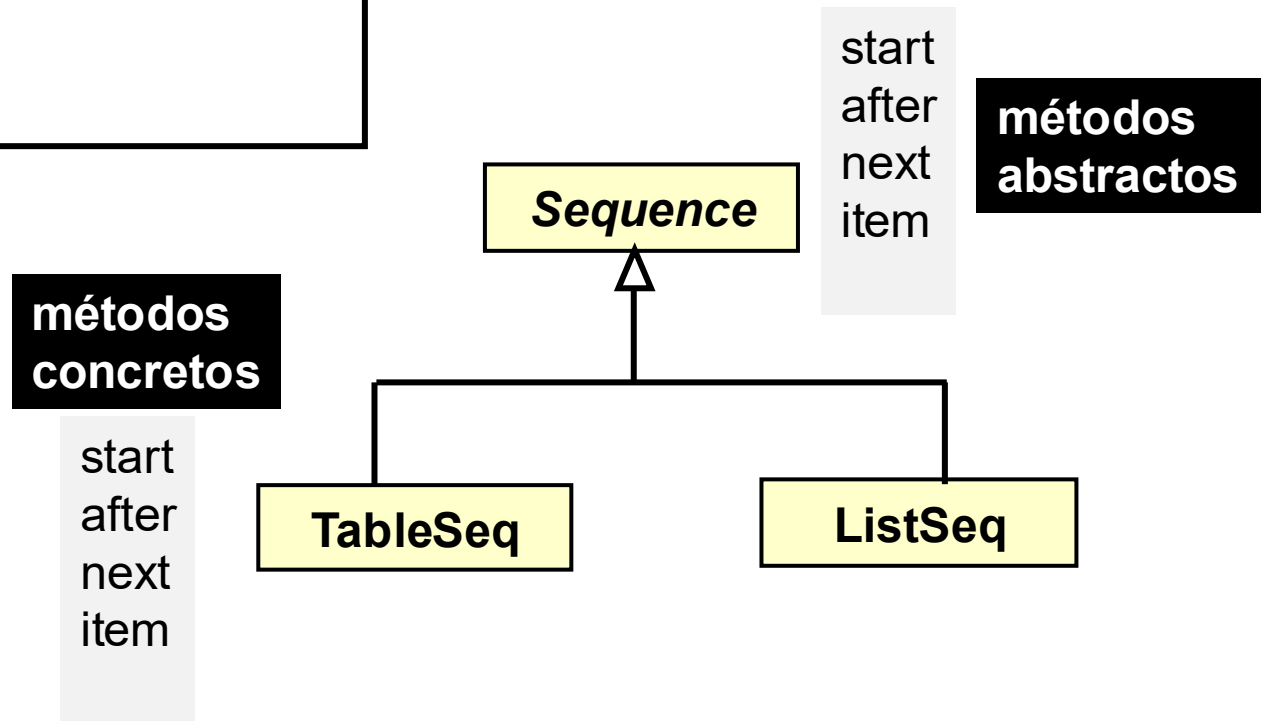
Motivación

- Esencial para escribir código en frameworks.
- Técnica fundamental para la reutilización en jerarquías de clases: factorizar comportamiento común.

Método Plantilla: Ejemplo de clase en lenguaje Eiffel

```
class Sequence<T>{..
    has(v:T): boolean {
        from start()
        until after() or else item().equal(v)
        loop next()
        end
        Result = not after()
    }
}
```

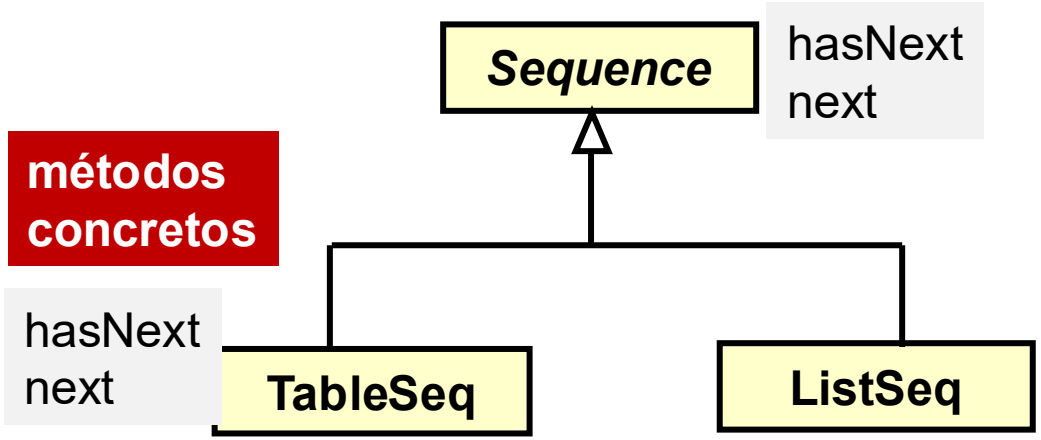
Esquema algorítmico de una búsqueda en una secuencia sin importar cómo esté implementada



Método Plantilla: Ejemplo Java

```
public class Sequence<T>{..
   public boolean has(T value) {
        while (hasNext()) {
             currentElement = next();
             if (currentElement.equals(value))
                  return true
        }
        return false;
    }
}
```

métodos abstractos



Método Plantilla: Aplicabilidad

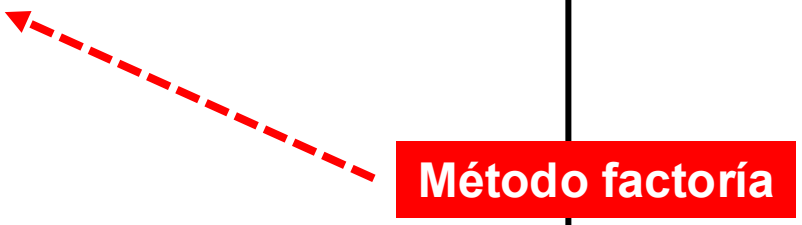
- Una clase implementa el **esquema de un algoritmo** y deja que las subclases implementen el comportamiento que puede variar.
- Cuando el **comportamiento común entre varias subclases** debe ser factorizado y localizado en una superclase común *(refactoring)*

Método Plantilla: Consecuencias

Un método plantilla (*template*) invoca a los siguientes tipos de métodos: operaciones abstractas y concretas en la clase abstracta o en clientes, métodos factoría, ..

Método Plantilla. Ejemplo *framework en DP*

```
// en clase abstracta Application de un framework
void openDocument (String nombre) {
    if (this.canOpenDocument(nombre)) {
        Document doc = this.createDocument();
        if (doc != null) {
                docs.addDocument(doc);
                aboutToOpenDocument(doc);
                doc.open();
                doc.read();
        }
    }
}
```



canOpenDocument(), createDocument() y aboutToOpenDocument() son **métodos abstractos** que se definen en las subclases

Método Plantilla. Ejemplo java.util

```
public abstract class AbstractCollection<E>
                                        implements Collection<E> {..

    public abstract Iterator<E> iterator();
    public abstract int size();

    public boolean contains(Object o) {
        Iterator<E> it = iterator();
        if (o==null) {
            while (it.hasNext())
                if (it.next()==null) return true;
        } else {
            while (it.hasNext())
                if (o.equals(it.next())) return true;
        }
        return false;
    }
}
```

Método Plantilla y expresiones lambda
("Java 8 Lambdas, Richard Warburton, 2014)

Una aplicación bancaria que gestiona préstamos a empresas, particulares y empleados.

```
public abstract class LoanApplication {

    public void checkLoanApplication() throws ApplicationDenied {
        checkIdentity();
        checkCreditHistory();
        checkIncomeHistory();
        reportFindings();
    }
    protected abstract void checkIdentity() throws ApplicationDenied;
    protected abstract void checkIncomeHistory() throws ApplicationDenied;
    protected abstract void checkCreditHistory() throws ApplicationDenied;
    private void reportFindings() {..}
}
```

Método plantilla invoca métodos abstractos

Método Plantilla basado en expresiones lambda

```
public class LoanApplication {
   private final Criteria identity;
   private final Criteria creditHistory;
   private final Criteria incomeHistory;

   public LoanApplication(Criteria identity,
                          Criteria creditHistory,
                          Criteria incomeHistory) {
        this.identity = identity;
        this.creditHistory = creditHistory;
        this.incomeHistory = incomeHistory;
   }
   public void checkLoanApplication()
                          throws ApplicationDenied {
        identity.check();
        creditHistory.check();
        incomeHistory.check();
        reportFindings();
   }
   private void reportFindings() {..}
}
```

Ejemplo anterior usando la interfaz funcional Criteria

```
public interface Criteria {
    public void check()
                throws ApplicationDenied;
}
```

Método Plantilla basado expresiones lambda

```
public class CompanyLoanApplication extends LoanApplication {
   public CompanyLoanApplication(Company company) {
        super(company::checkIdentity,
              company::checkHistoricalDebt,
                company::checkProfitAndLoss);
   }
}
```

- La subclase no necesita implementar métodos, son implementados en las clases `Company, Person y Employee.`
- `Más flexibilidad.`

Estrategia (Strategy)

Propósito

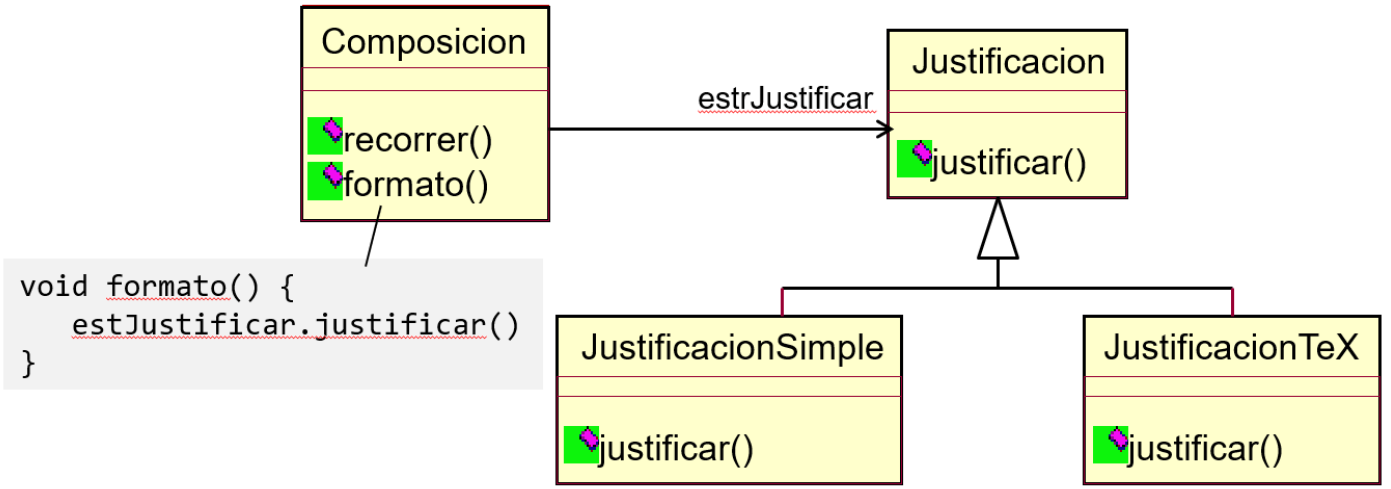
Define una **familia de algoritmos**, encapsula cada uno, y permite intercambiarlos. Permite variar los algoritmos de forma independiente a los clientes que los usan

Motivación

- Existen muchos algoritmos para justificar un texto, cómo no acoplar la aplicación a uno de ellos.
- Cómo implementar las políticas de descuento en una aplicación de ventas.

Estrategia. Ejemplo “Justificación de texto”

Ejemplo DP, GoF

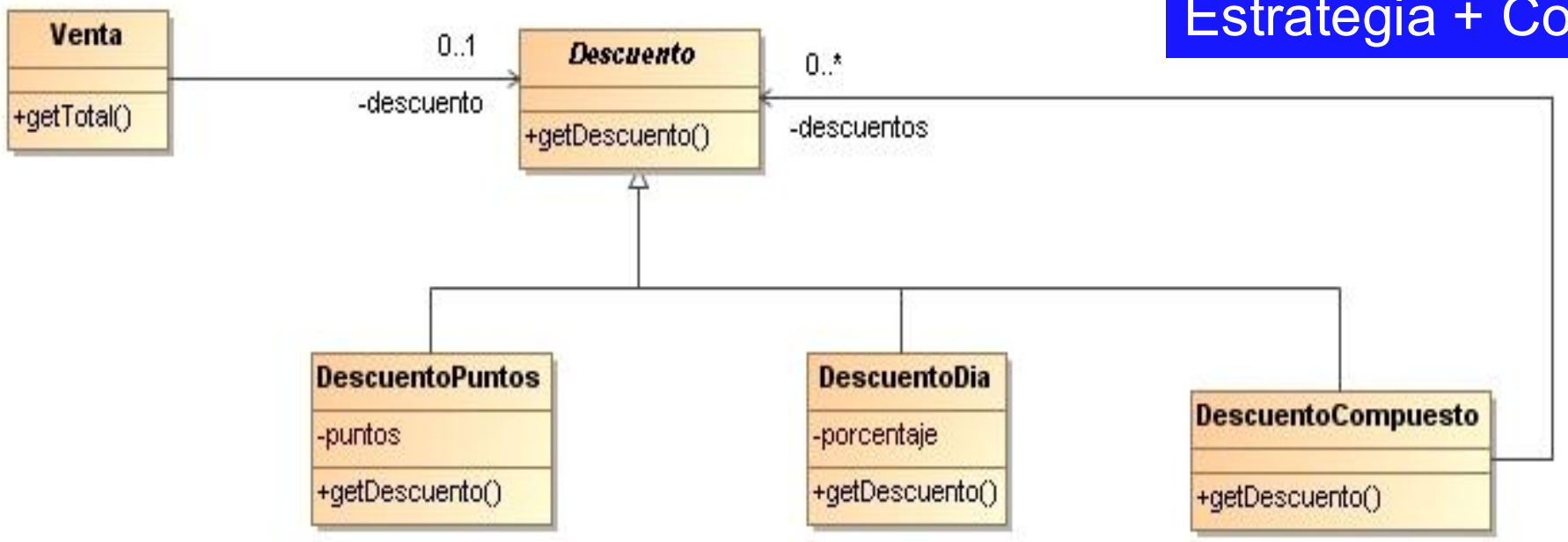



```
void formato() {
   estJustificar.justificar()
}
```

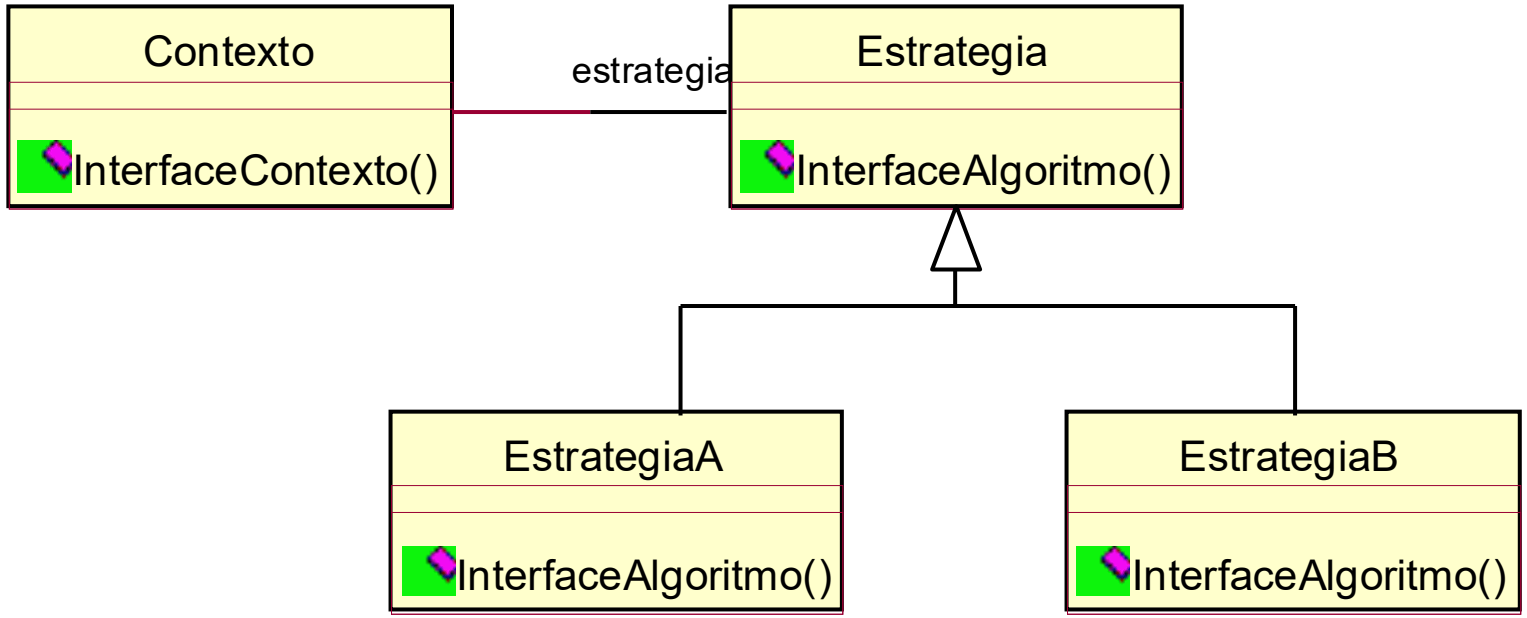
Estrategia.
Ejemplo "Política de Descuentos"

UML y Patrones, Craig Larman

Estrategia + Composite



Estrategia. Estructura



Estrategia: Implementación

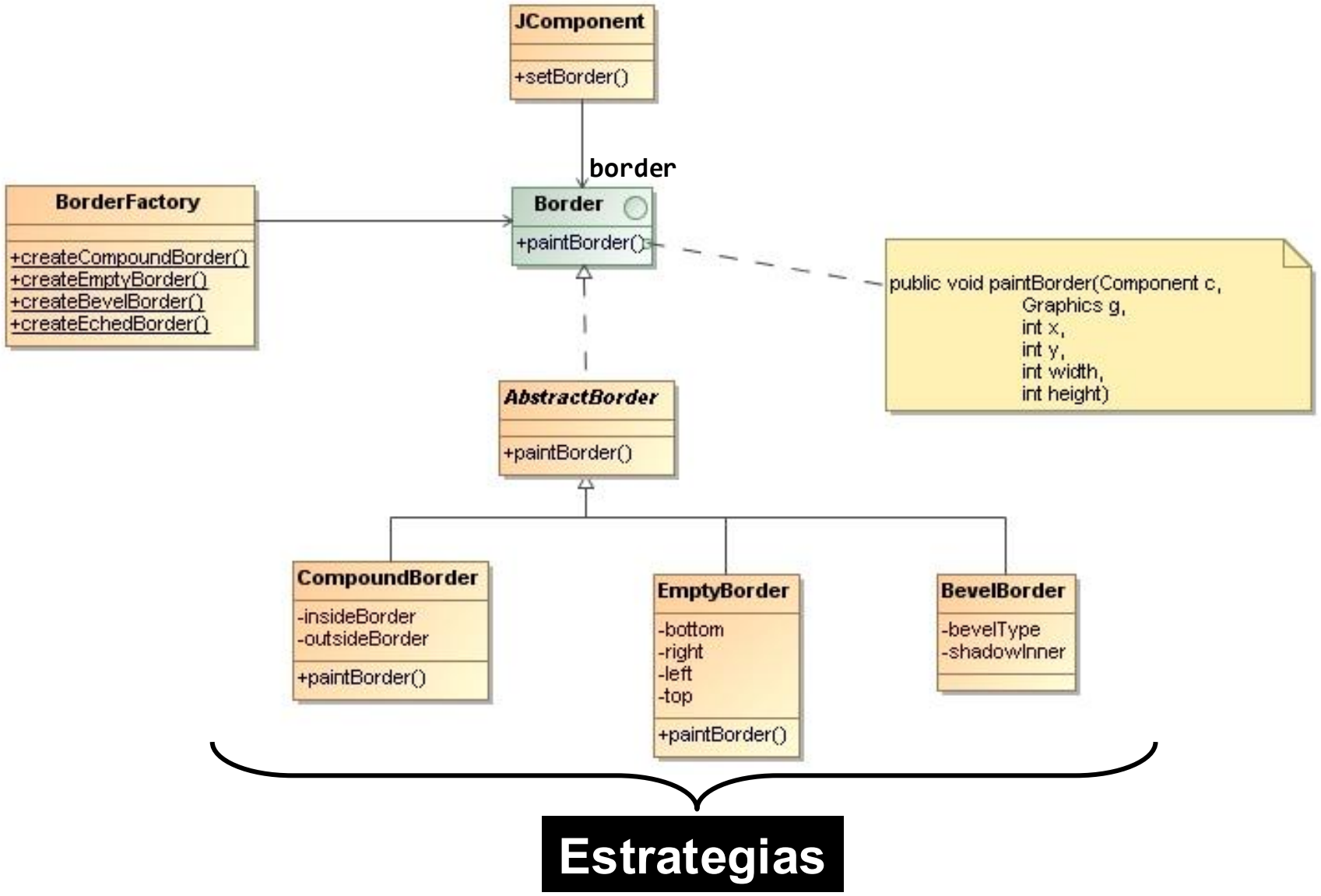
- **¿Cómo una estrategia concreta accede a los datos del contexto?**
  - Pasar datos o un objeto Contexto como argumentos
  - *Estrategia* almacena una referencia al contexto
- **¿Cómo se crea una instancia de una estrategia concreta?**
  - Uso de una **factoría**

Estrategia

- **Aplicabilidad**
  - Configurar una clase con uno de **varios comportamientos** posibles.
  - Se necesitan **diferentes variantes de un algoritmo.**
  - Una clase define muchos comportamientos que aparecen como **sentencias *case*** en sus métodos (refactoring).

- **Consecuencias**
  - Define una familia de algoritmos relacionados.
  - Una alternativa a crear subclases de la clase *Contexto*.
  - Elimina sentencias *case*
  - En el código cliente se puede elegir entre diferentes estrategias.
  - ***Estado*** y ***Estrategia*** son similares, cambia el *Propósito*: ejemplos de composición con delegación

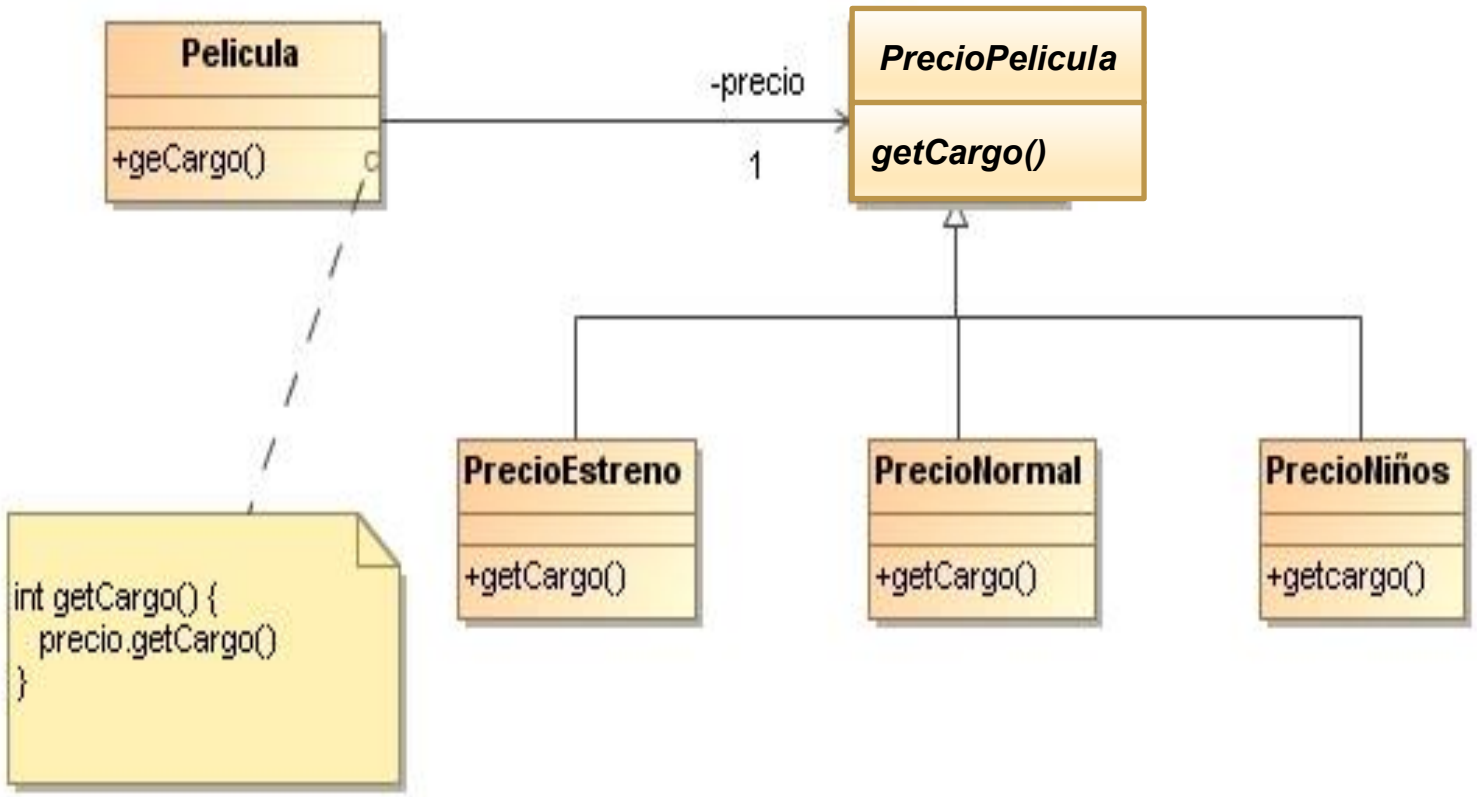
Estrategia. Ejemplo de Bordes en Swing



Cambiar switch por Estrategia (Refactoring en clase prácticas)

```
cantidadTotal = 0;
for(Alquiler alq:_alquileres) {
        double unaCantidad = 0;
        //determinar cantidad para cada linea
        switch (alq.getPelicula().getCodigoPrecio()) {
          case Pelicula.NORMAL:
               unaCantidad += 2;
               if (alq.getDiasAlquiler() > 2)
                      unaCantidad += (alq.getDiasAlquiler()-2)*1.5;
               break;
          case Pelicula.ESTRENO:
               unaCantidad += alq.getDiasAlquiler() * 3;
               break;
          case Pelicula.NIÑOS:
               unaCantidad += 1.5;
               if (alq.getDiasAlquiler() > 3)
                      unaCantidad += (alq.getDiasAlquiler()-3)*1.5;
               break;
        }
       cantidadTotal = cantidadTotal + unaCantidad;
   }
```

Cambiar `switch` por Estrategia (Refactoring en clase prácticas)



Cambiar `switch` por Estrategia (Refactoring en clase prácticas)

```
public class Pelicula {...
    private String _titulo;
    private PrecioPelicula _precio;
    public Pelicula(String titulo, String tipo) {
        _titulo = titulo;
        try {
            _precio = (PrecioPelicula)
                    Class.forName("videoclubFowler.precioPelicula“
                                + tipo.toUppercase()).newInstance();
        } catch(Exception e) {
            e.printStackTrace();
        }
    }
```

```
public double getCosteAlquilerPelicula(Alquiler alquiler) {
        return _precio.getCargo(alquiler);
}
```

Estrategia y Expresiones Lambda: Ejemplo “Precios Viaje”

```
public interface ReglaPrecioViaje {
    public double getPrecio(Viaje viaje);
}
```

interfaz funcional para aplicar la estrategia

```
public class Viaje {
    int distancia;
    Confort tipoCoche;
    ReglaPrecioViaje reglaPrecio;

    public double calcularPrecio() {
        return reglaPrecio.getPrecio(this);
    }
    public void setReglaPrecio(ReglaPrecioViaje regla) {
        this.reglaPrecio = regla;
    }
    // métodos getters y setters para distancia y tipoCoche
}
```

Estrategia con expresiones lambda

```
public class App {
    public static void main(String[] args) {
        Viaje viaje = new Viaje();
        viaje.setDistancia(500);
        viaje.setTipoCoche(Confort.MEDIO);
        ReglaPrecio regla = (Viaje v) -> {
                double precio;
                switch (v.getTipoCoche()){
                    case BAJO: precio = 5;break;
                    case MEDIO: precio = 10;break;
                    case ALTO: precio = 15;break;
                    default: precio =0;
                }
                precio = precio + v.getDistancia()*0.1;
                return precio;
         };
        viaje.setReglaPrecio(regla);
        double precio = viaje.calcularPrecio();
        System.out.println(precio);
    }
}
```

¿Cuándo interesa usar expresiones lambda en Estrategia?

Cómo implementar y registra las reglas en una aplicación

¿Cuándo interesa usar expresiones lambda en Estrategia?

- Las implementaciones son simples y no requieren mantener un estado.
- Las estrategias cambian con frecuencia, con lambdas no hay que añadir o modificar clases.

¿Cómo obtener una regla o estrategia concreta?

```
(1) Un map de estrategias implementadas como expresiones
  lambdas de un tipo de interfaz funcional.
(2) Un map de suppliers que retornan un objeto de una
  estrategia concreta
(3) Usando introspección
```

Estado (State)

Propósito

Permite a un objeto cambiar su comportamiento cuando cambia su estado. El objeto parece cambiar de clase.

Motivación

- Una *conexión TCP* puede encontrarse en uno de varios estados, y dependiendo del estado responderá de un modo diferente a los mensajes de otros objetos para solicitudes tales como abrir, cerrar o establecer conexión.
- Un *cajero automático* realiza sus operaciones dependiendo de su estado interno: *disponible*, *sin dinero*, *sin papel*, *procesando trabajo interno*, *sin conexión*,…
- Una *máquina vendedora* de productos también tiene diferentes estados.
- Un termostato inteligente para el control de la climatización en una casa domótica

Noción de **estado de un objeto**: "*Un objeto se encuentra en una situación en la que se **satisface una condición** o está a la espera de que se produzca un **evento***"

Estado: Ejemplo Máquina Expendedora

```
public  class  SnackVendingMachine {
        final  static  int  LISTA  = 0;
        final  static  int  EXPENDIENDO   = 1;
        final  static  int  SIN_CAMBIO   = 2;
        final  static  int  INSERTAND0_MONEDAS    = 3;
        final  static  int  PRODUCTO_SERVIDO    = 4;
        final  static  int  FUERA_DE_USO   = 5;

        private  int  estado  = LISTA ;

        private  Product[][]  products ;

        public  SnackVendingMachine()..}

        public  boolean  insertarMoneda( int cant){

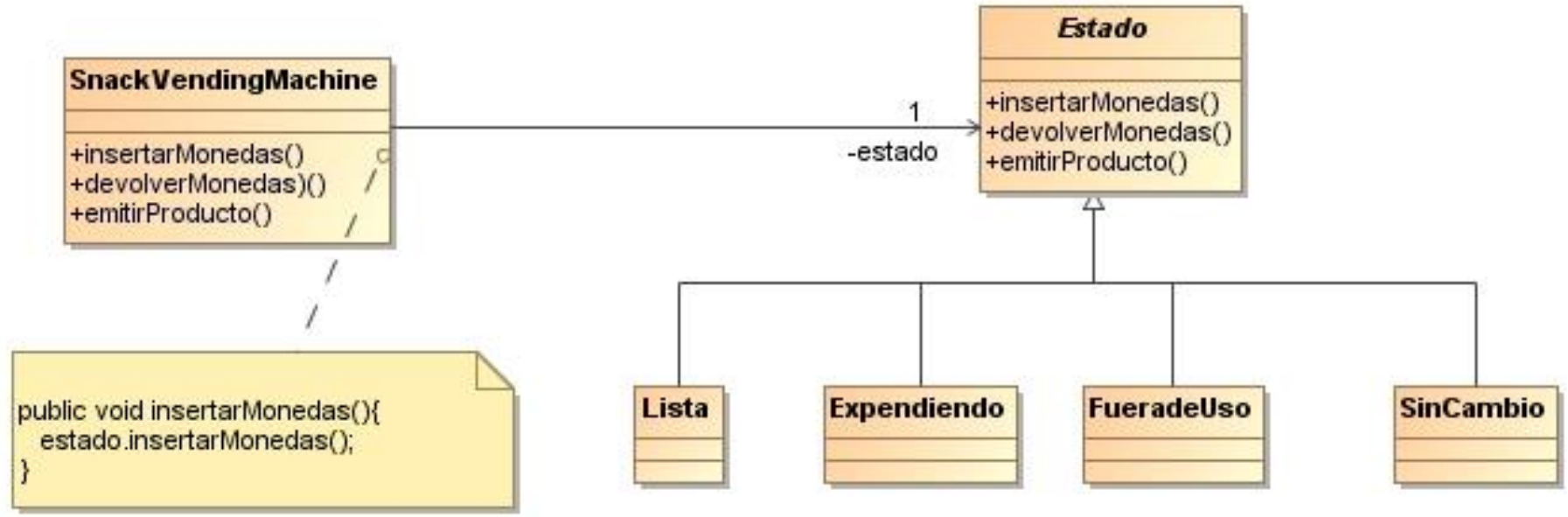
                if (estado  == LISTA ) { ..

                } else  if (estado  == EXPENDIENDO  ) { ...
                } else  if (estado  == FUERA_DE_USO  ) { ...
                } else  if (estado  == SIN_CAMBIO  ) { ...  }
        }
        public  boolean  devolverMonedas() {
                if (estado  == INSERTANDO_MONEDAS   ) {..
                } else  if (estado  == EXPENDIENDO  ) {..
                } else  if (estado  == FUERA_DE_USO  ) {..
                } else  if (estado  == LISTA ) {...}
        }
        // otros métodos cuya implementación también está basada en un análisis
       // de casos sobre el atributo estado

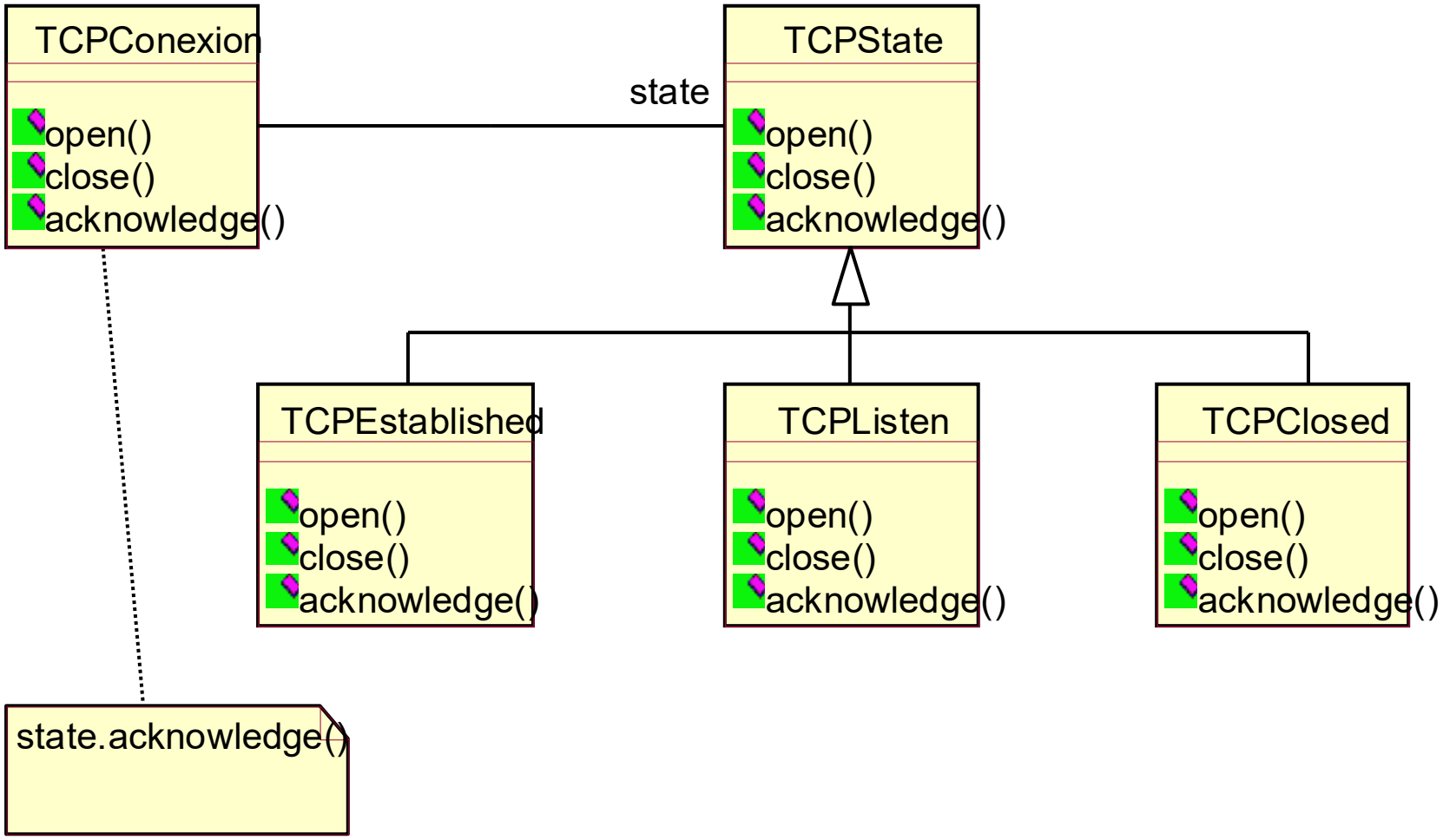
}
```

Estado: Ejemplo Máquina Expendedora

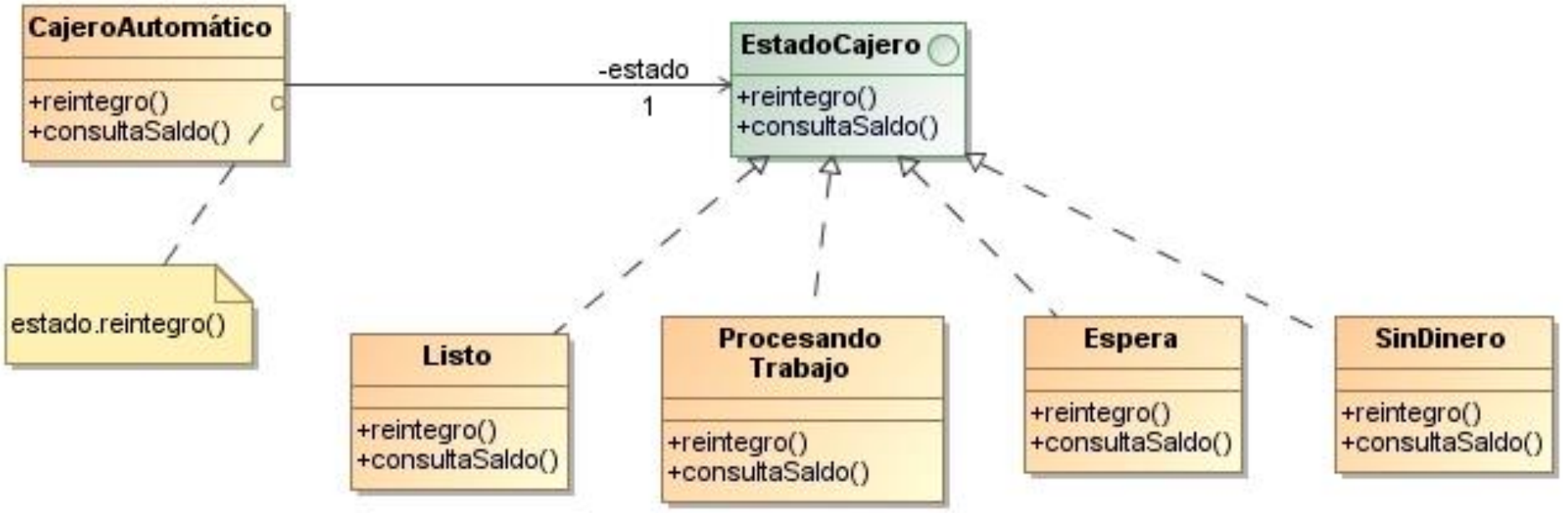
Eliminar CASE con una jerarquía de estados



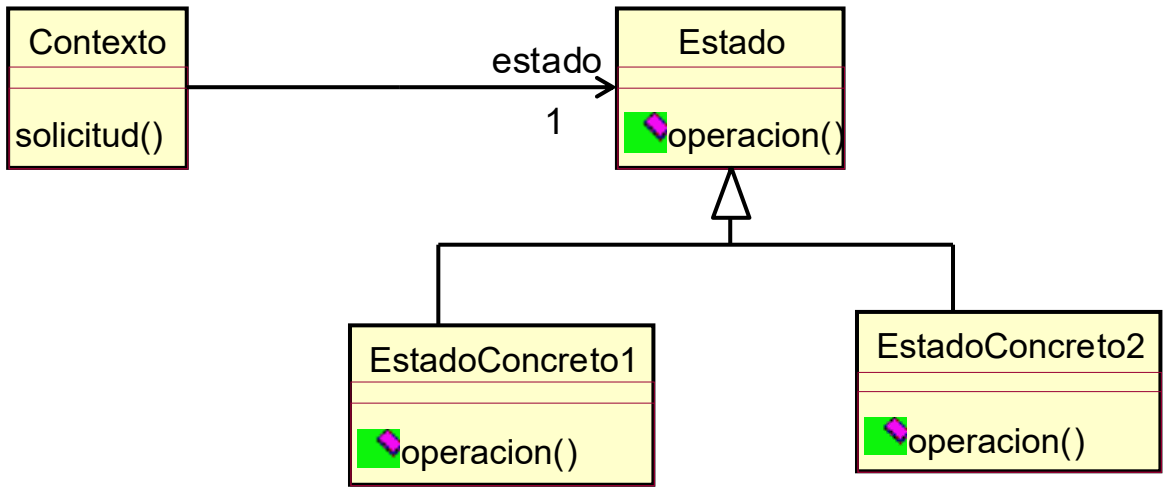
Estado. Ejemplo Conexión TCP (en DP-GoF)



Estado. Ejemplo “Cajero Automático”



Estado: Estructura



Estado: Aplicabilidad

- El comportamiento del objeto depende de su estado, y debe cambiar su comportamiento en tiempo de ejecución dependiendo de su estado.
- Las operaciones tienen grandes estructuras CASE que dependen del estado del objeto, que es representado por uno o más constantes de tipo enumerado (refactoring).

Estado: Implementación

- **¿Quién define las transiciones entre estados?** Contexto o subclases estado. Más apropiado que sean los estados y Contexto ofrece un método para ello.
- **¿Cuándo son creados los objetos estado?**
  - Pueden crearse cuando se necesiten o con antelación y que Contexto tenga una referencia a ellos.
  - Los objetos *estado* pueden ser ***Singleton***.

Estado: Consecuencias

- Todo el comportamiento asociado a un particular estado es **embebido en una clase**.
- **Subclases en vez de Sentencias CASE**.
- Ayuda a **evitar estados inconsistentes** en caso de estado representado por varias variables.
- **Transiciones de estado son más explícitas**.
- Incrementa el número de objetos.

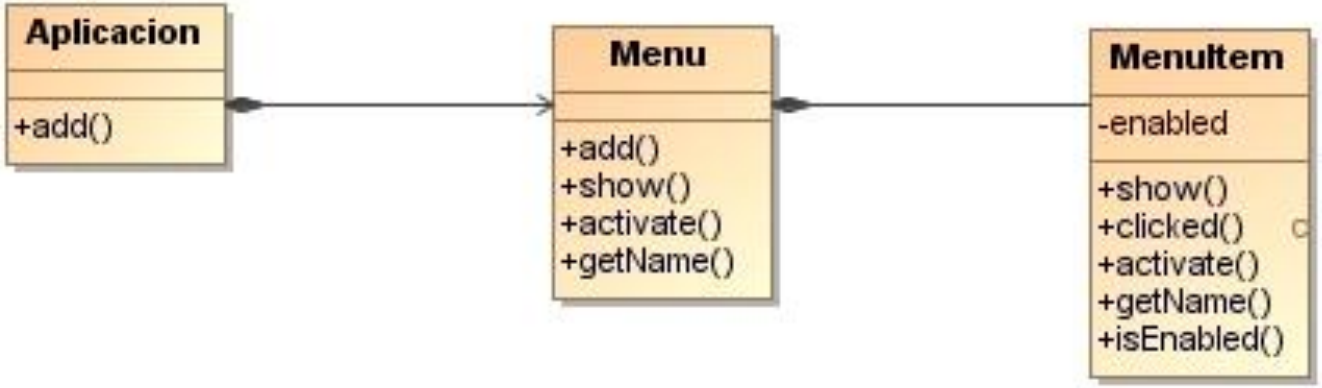
Command (Orden)

Propósito

Encapsula un mensaje como un objeto, permitiendo **parametrizar** métodos y objetos con mensajes, añadir mensajes a una cola y soportar funcionalidad ***deshacer/rehacer* (*undo/redo*)**

Motivación

- Algunas veces es necesario **enviar un mensaje** a un objeto **sin conocer el selector del mensaje ni el objeto receptor.**
- Por ejemplo, *widgets* como botones o menús deben ejecutar una acción cuando el usuario hace clic sobre ellos.

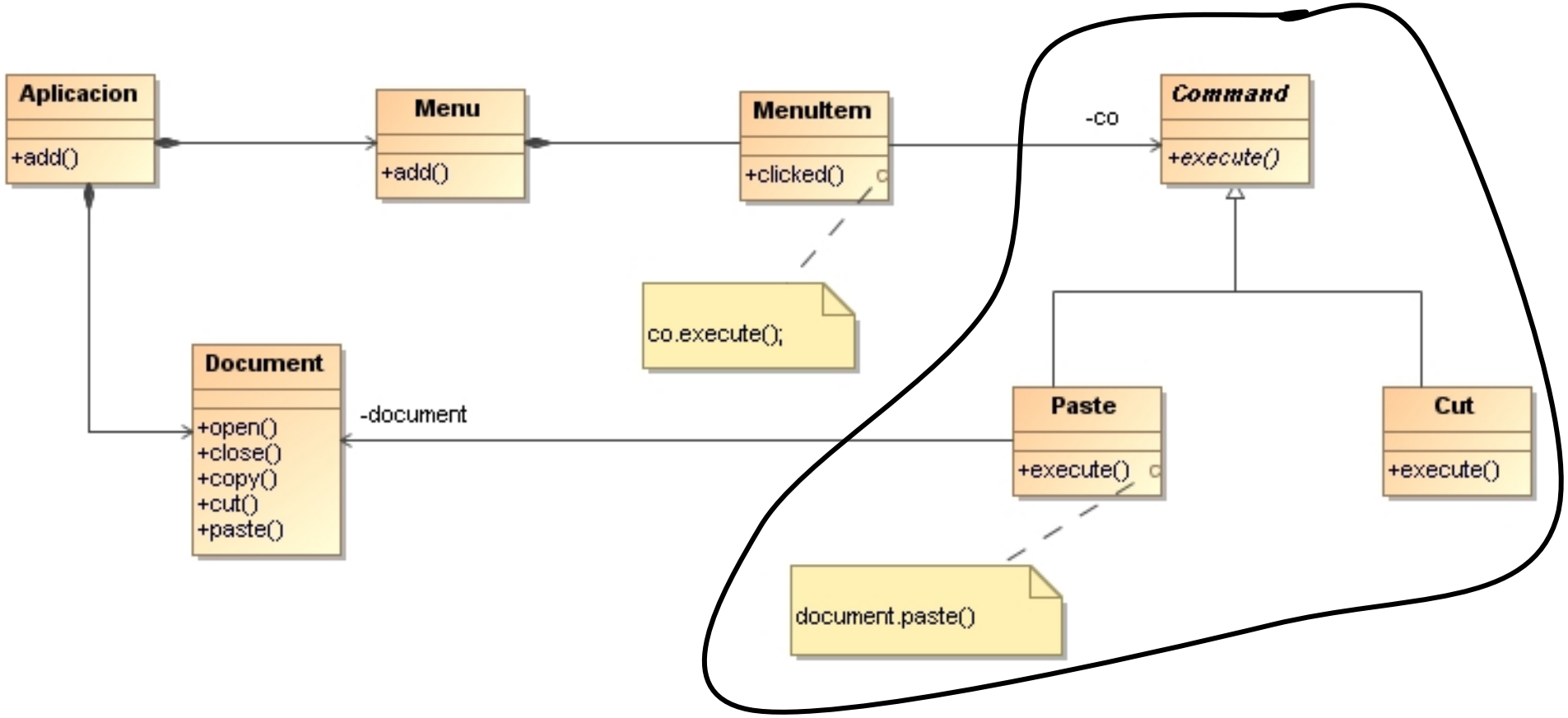


¿Cómo parametrizar `MenuItem` con la acción que debe ejecutarse?

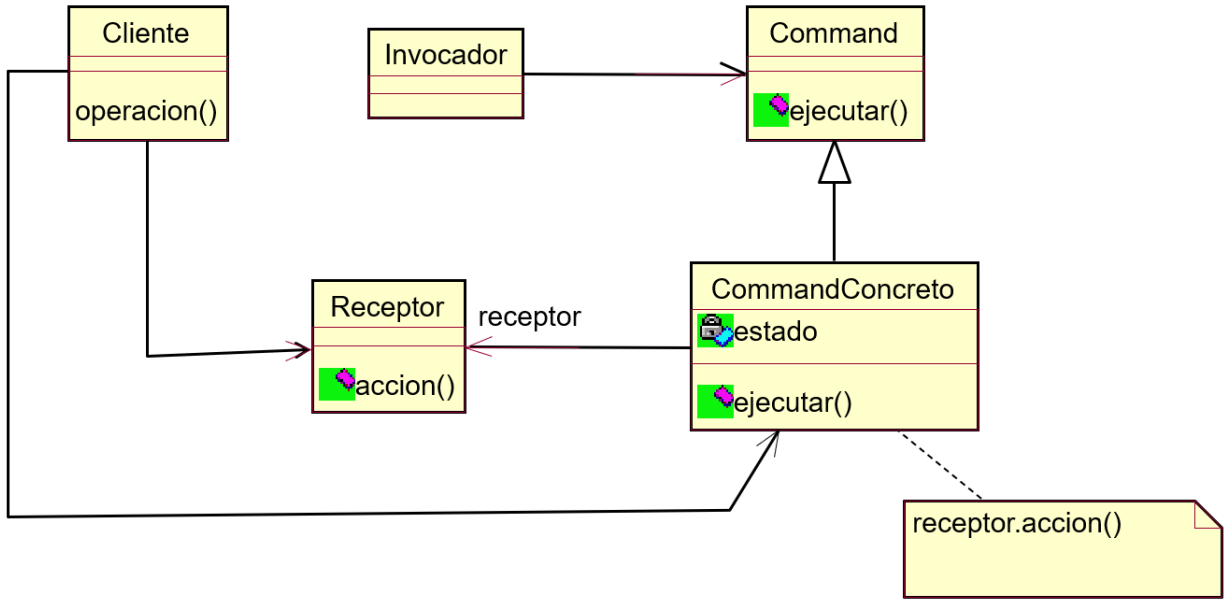
Command: Motivación

¿Cómo parametrizar `MenuItem` con la acción que debe ejecutarse?

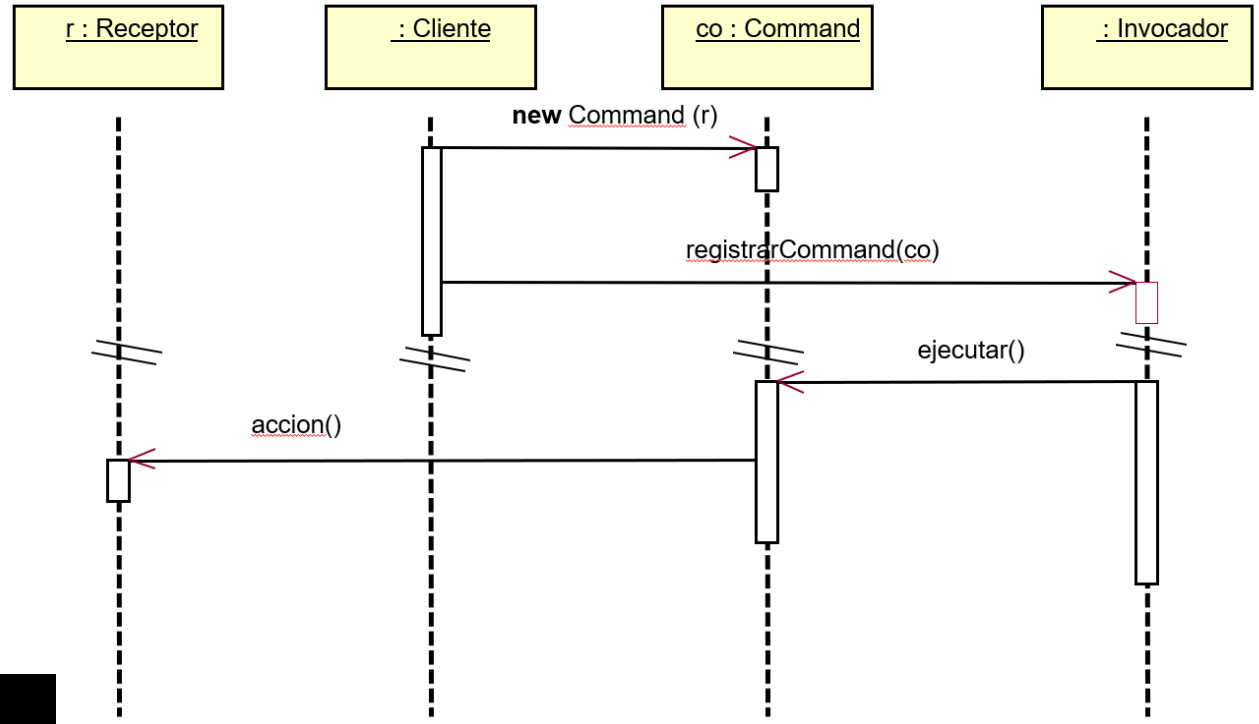
Lenguaje sin lambdas ni metaprogramación, p.e., Java < 8



Command: Estructura



Command: Comportamiento



Command: Implementación

En lenguajes que soportan expresiones lambda (Java, Python, Scala,…) , punteros a funciones (C++) o metaprogramación, el patrón *Command* sólo **sería útil para soportar *undo/redo.***

Command: Aplicabilidad

- **Parametrizar objetos** por la acción a realizar cuando el lenguaje no soporta pasar código como parámetro de métodos:
  - En C++ se puede usar punteros a funciones -> mecanismo ***Callback:*** *función que es registrada en el sistema para ser llamada más tarde;*
  - Java 8, Python, Scala, Smalltak**,..,** incluyen **bloques de código** (expresiones lambda) **e introspección.**

- Añadir a una cola y ejecutar mensajes después de su ejecución
  - Un objeto Command tiene un tiempo de vida independiente de la solicitud original, por ejemplo para soportar **facilidad undo/redo** o implementar un mecanismo de **recuperación de fallos**.

Command en Java con expresiones lambda (sin considerar que ya existe la interfaz Consumer)

```java
// interfaz funcional
public interface Command {
    public void perform();
}
```

```java
public class MenuItem {
    public void clicked(Command co) {
        co.perform();
    }
}
```

```java
public class App {

    public static void main(String[] args) {
        Cuenta c1 = new Cuenta (800000, "123", "Juan");

        MenuItem item1 = new MenuItem();
        item1.clicked(() -> System.out.println(c1.getSaldo()));
    }
}
```

Command en Java con expresiones lambda y generics (sin considerar interfaz Consumer)

```java
public interface Command<T> {
     public void perform(T t);
}
```

```java
public class MenuItem<T> {
    Command<T> action;
    T obj;
    public MenuItem(Command<T> action, T obj){
         this.action = action;
         this.obj= obj;
    }
    public void clicked(){
         action.perform(obj);
    }
}
```

```java
public class App {

    public static void main(String[] args) {
         Cuenta cuenta = new Cuenta (800000, "123", "Juan");
         Command<Cuenta> action = c-> System.out.println(c.getSaldo());
         MenuItem<Cuenta> item1 = new MenuItem<Cuenta>(action,cuenta);
         item1.clicked();
    }
}
```

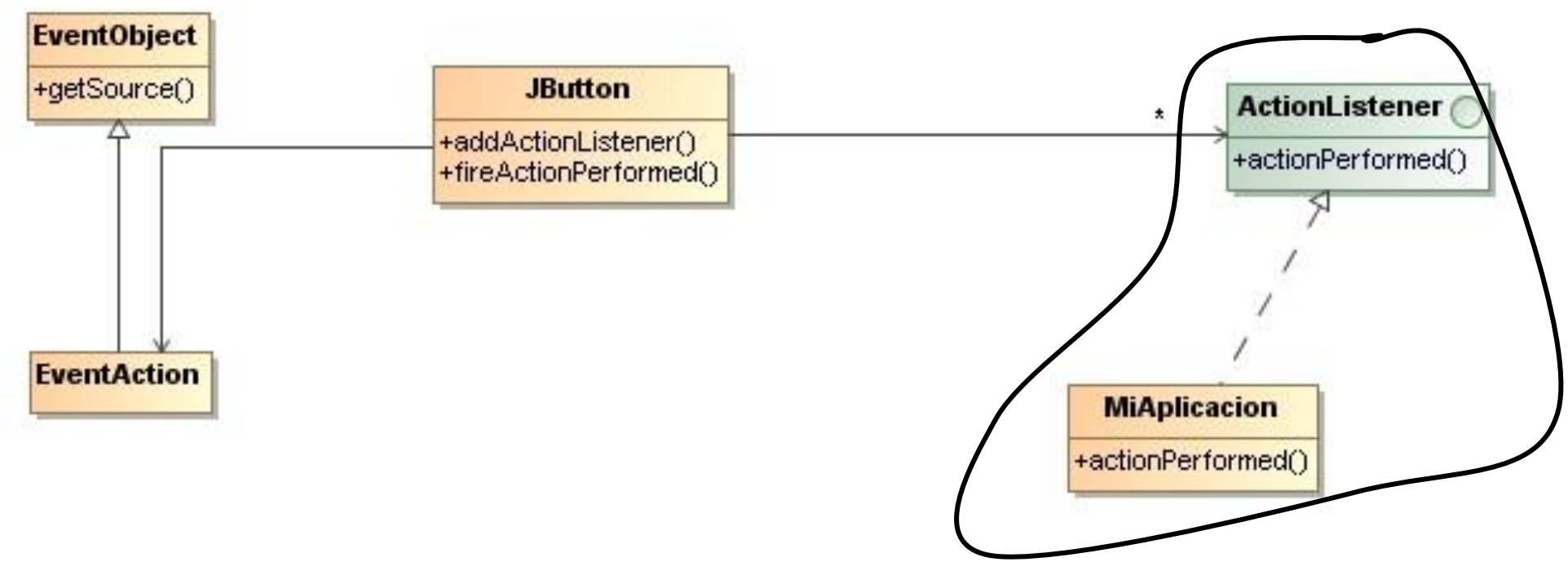
Command con interfaz Consumer

```
public class MenuItem<T> {
    Consumer<T> action;
    T obj;

    public MenuItem(Consumer<T> action, T obj){
        this.action = action;
        this.obj= obj;
    }
    public void clicked(){
        action.accept(obj);
    }
}
```

```
public class App {
    public static void main(String[] args) {
        Cuenta c1 = new Cuenta (800000, "123", "Juan");
        Consumer<Cuenta> action =  c->
                                System.out.println(c.getSaldo());
        MenuItem<Cuenta> item1 = new MenuItem<Cuenta>(action,c1);
        item1.clicked();
    }
}
```

Button en Swing



`ActionListener` juega el papel de un **Command** pero veremos que es parte de una implementación del patrón **Observer**.

Command: Consecuencias

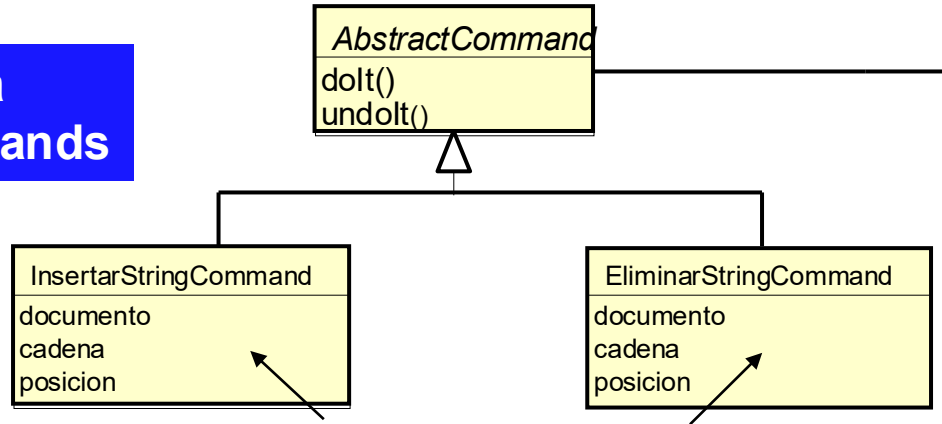
- Desacopla el objeto que invoca la operación del objeto que sabe cómo realizarla.
- Cada subclase *CommandConcreto* encapsula un par receptor/acción, almacenando el receptor como un atributo e implementando el método *ejecutar*.
- Objetos *command* pueden ser **manipulados como cualquier otro objeto.**
- Se pueden crear *command* compuestos (aplicando el patrón *Composite*).

Command: Undo/Redo

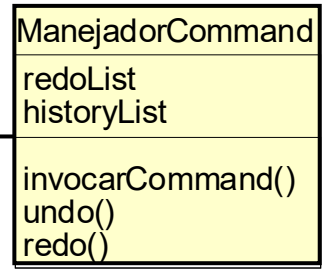
- Una jerarquía de *Commands* representa las operaciones que se pueden deshacer/rehacer.
- Los *command* deben tener atributos para registrar la información que permita deshacer/rehacer.
- Una lista "historia" de objetos *commands registra copias de las operaciones realizadas (una única lista doblemente enlazada o tener dos listas: historia y redo)*
- Una clase "manager" soporta el manejo de las listas de *commands* y de la recepción de operaciones.
- Las operaciones ***undo*** y ***redo*** se tratan de **forma distinta** al resto de *commands*.

Command. Mecanismo Undo/Redo
(extraído de “Patterns in Java”, Mark Grand)

Jerarquía de commands

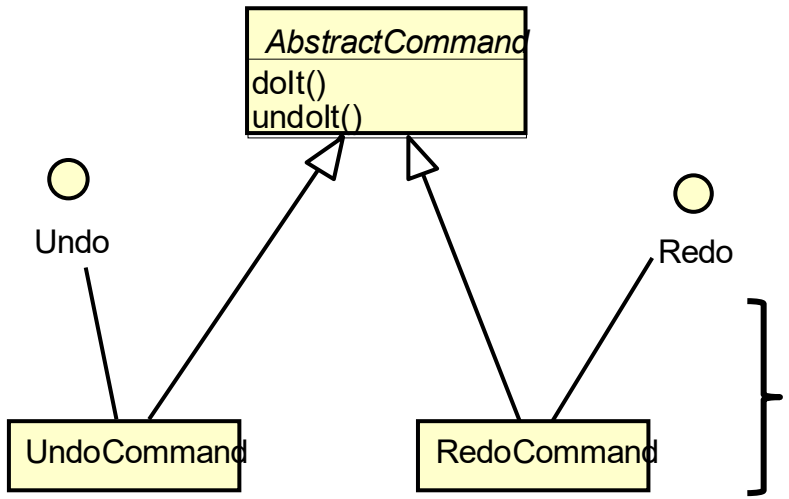


Manejador de commands con 2 LinkedLists



info para undo/redo

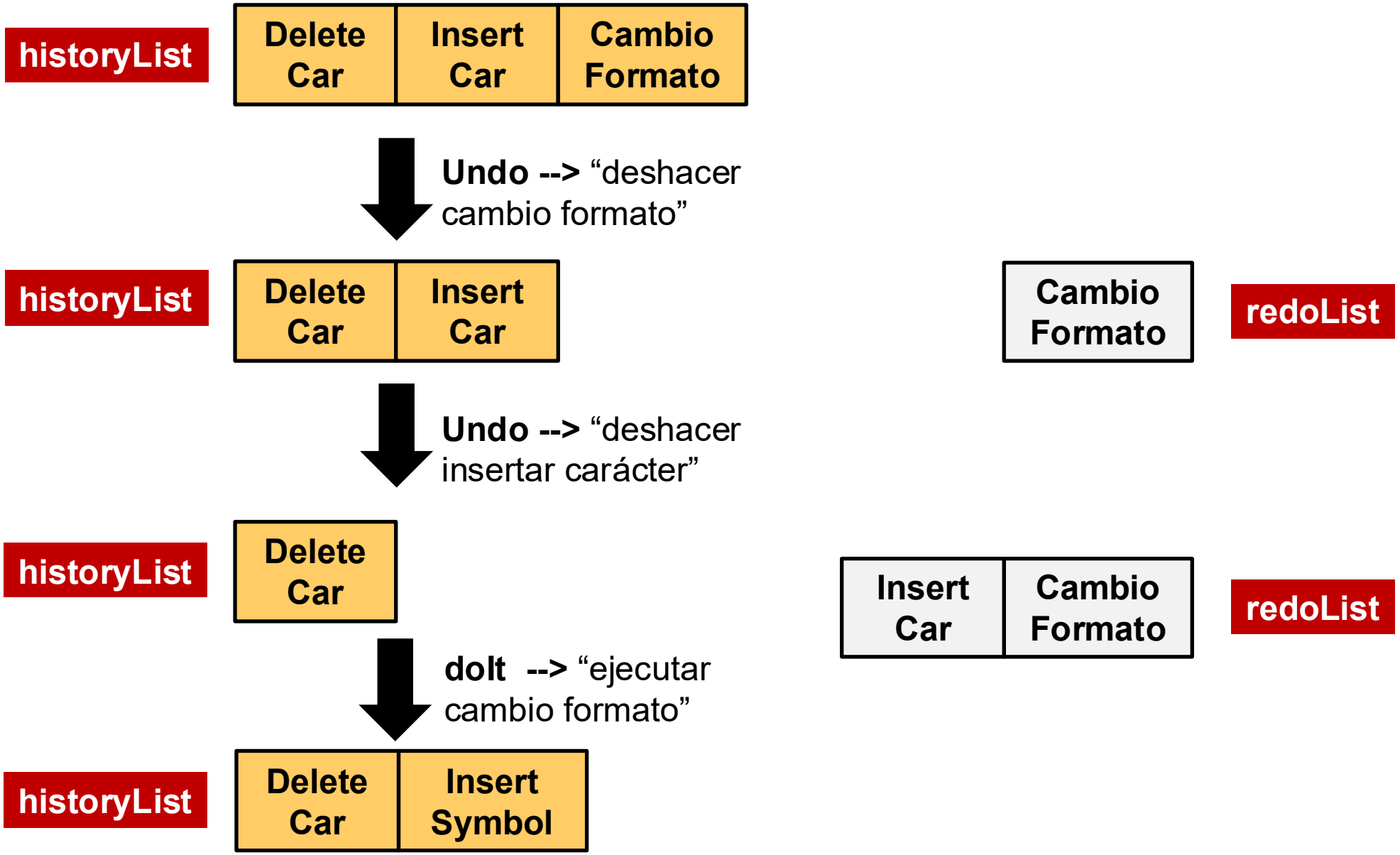
```
public class InsertarStringCommand {
    public void doIt(){
        documento.insertarString(posicion, cadena);
    }
    public void undoIt(){
        documento.eliminarString(posicion, cadena.length());
    }
}
```



Undo y redo son interfaces marcadoras sin métodos a implementar

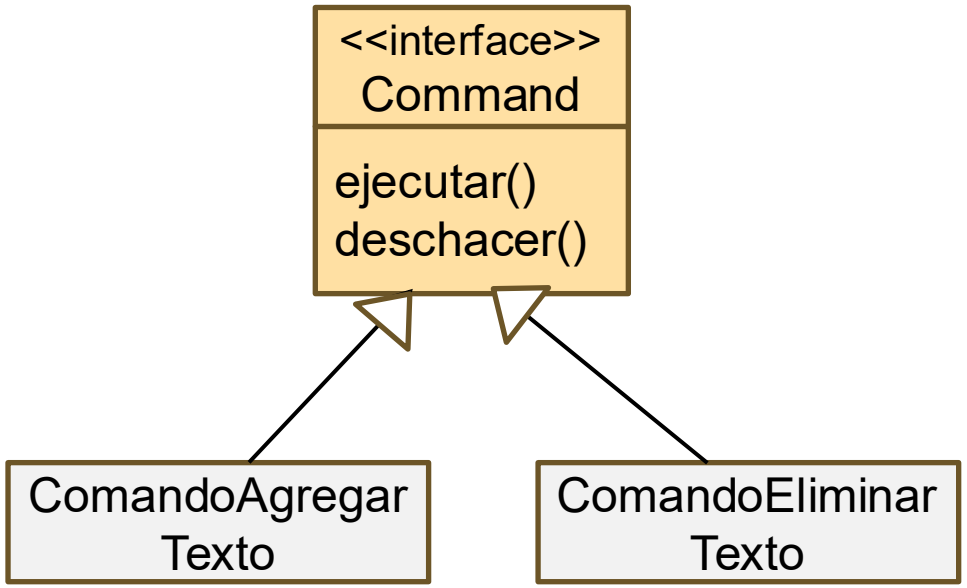
Command. Mecanismo Undo/Redo

```
public class ManejadorCommand {..
    private LinkedList<AbstractCommand> historyList;
    private LinkedList<AbstractCommand> redoList;
    public void invocarCommand (AbstractCommand command){
        if (command instanceof Undo)
            undo(); return;
        if (command instanceof Redo)
            redo(); return;
        if (command.doIt())
              addToHistory(command);
        else historyList.clear();
        if (redoList.size() > 0) redoList.clear();
    }
    private void undo() {
        if (historyList.size() > 0) {
            AbstractCommand undoCmd = historyList.removeFirst();
            undoCmd.undoIt();
            redoList.addFirst(undoCmd);
        }
    }
    private void redo() {
}
```


historyList
Delete Car
Insert Car
Cambio Formato
Undo --> "deshacer cambio formato"
historyList
Delete Car
Insert Car
Cambio Formato
redoList
Undo --> "deshacer insertar carácter"
historyList
Delete Car
Insert Car
Cambio Formato
redoList
doIt --> "ejecutar cambio formato"
historyList
Delete Car
Insert Symbol

Command. Mecanismo Undo/Redo
Ejemplo generado por ChatGPT 4o

```
public interface Command {
    void ejecutar();  // Ejecutar el comando
    void deshacer();  // Deshacer el comando
}
```




```
public class EditorTexto {
    private StringBuilder texto =
                        new StringBuilder();
    public void agregarTexto(
                        String nuevoTexto) {
        texto.append(nuevoTexto);
    }
    public void eliminarTexto(int longitud) {
        if (longitud <= texto.length()) {
            texto.delete(texto.length() -
            longitud, texto.length());
        }
    }
    public String obtenerTexto() {
        return texto.toString();
    }
}
```

Command. Mecanismo Undo/Redo
Ejemplo generado por ChatGPT 4o

```
public class ComandoAgregarTexto implements Command {
    private EditorTexto editor;
    private String textoAgregar;

    public ComandoAgregarTexto(EditorTexto editor, String textoAgregar) {
        this.editor = editor;
        this.textoAgregar = textoAgregar;
    }
    @Override
    public void ejecutar() {
        editor.agregarTexto(textoAgregar);
    }
    @Override
    public void deshacer() {
        editor.eliminarTexto(textoAgregar.length());
    }
}
```

Command. Undo/Redo

Ejemplo generado por ChatGPT 4o

```java
public class GestorComandos {
    private Deque<Command> pilaUndo = new ArrayDeque<>();
    private Deque<Command> pilaRedo = new ArrayDeque<>();

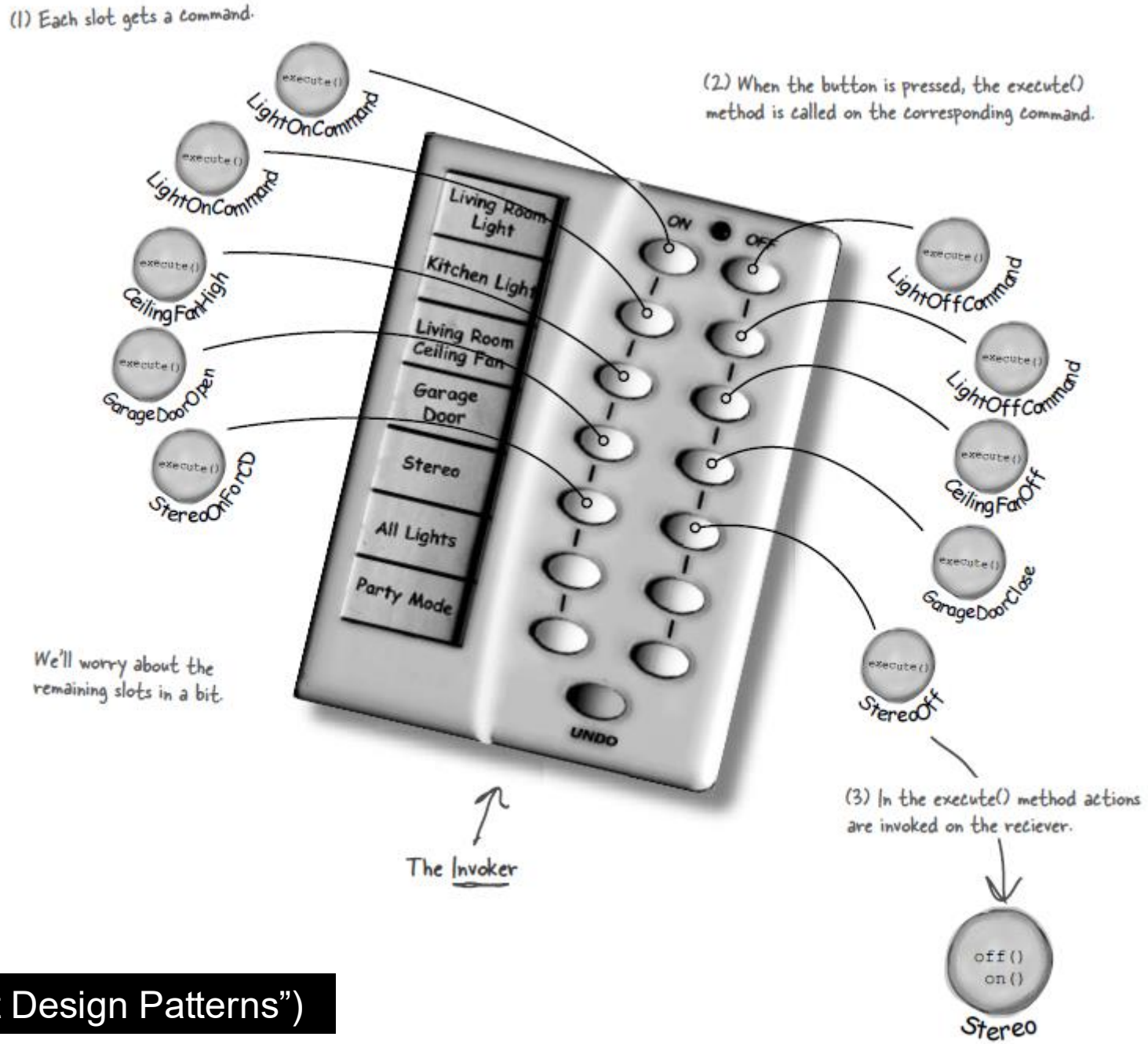
    public void ejecutarComando(Command comando) {
        comando.ejecutar();
        pilaUndo.push(comando);
        pilaRedo.clear();
    }
    public void undo() {
        if (!pilaUndo.isEmpty()) {
            Command comando = pilaUndo.pop();
              comando.deshacer();
            pilaRedo.push(comando);
        } else {
            System.out.println("No hay acciones para deshacer.");
        }
    }
    public void redo() {
        if (!pilaRedo.isEmpty()) {
            Command comando = pilaRedo.pop();
            comando.ejecutar();
            pilaUndo.push(comando);
        } else {
            System.out.println("No hay acciones para rehacer.");
        }
    }
}
```

Command. Ejercicio “Mando de Control Remoto” (extraído de “Head First Design Patterns”)

Una empresa desea construir una API para un **mando de control remoto** programable de casas inteligentes. El mando podrá ser programado con **7 funciones** diferentes y tendrá para cada una de ellas **dos botones**, uno de encendido (on) y otro de apagado (off). Cada función puede ser asignada a uno de los **dispositivos controlables** existentes en la casa (luces, termostato, puerta del garaje, ventilador techo, TV, etc.). El mando también tiene un botón que permite **“deshacer”** la acción ejecutada por el último botón presionado. Para cada dispositivo existe una clase que implementa su control (por ejemplo, las clases `Luz`, `VentiladorTecho`, `PuertaGaraje`, etc.) que son proporcionadas por los fabricantes.

a) **Diseña una solución basada en patrones de diseño para el software de control del mando** teniendo en cuenta que debe poder controlar dispositivos ya existentes u otros que puedan aparecer en el futuro. Dibuja el diagrama de clases y escribe el código Java de la solución.

b) Modifica la solución anterior para que sea posible asignar el control de varios dispositivos a un botón del mando --> **¿Cómo se consigue?**

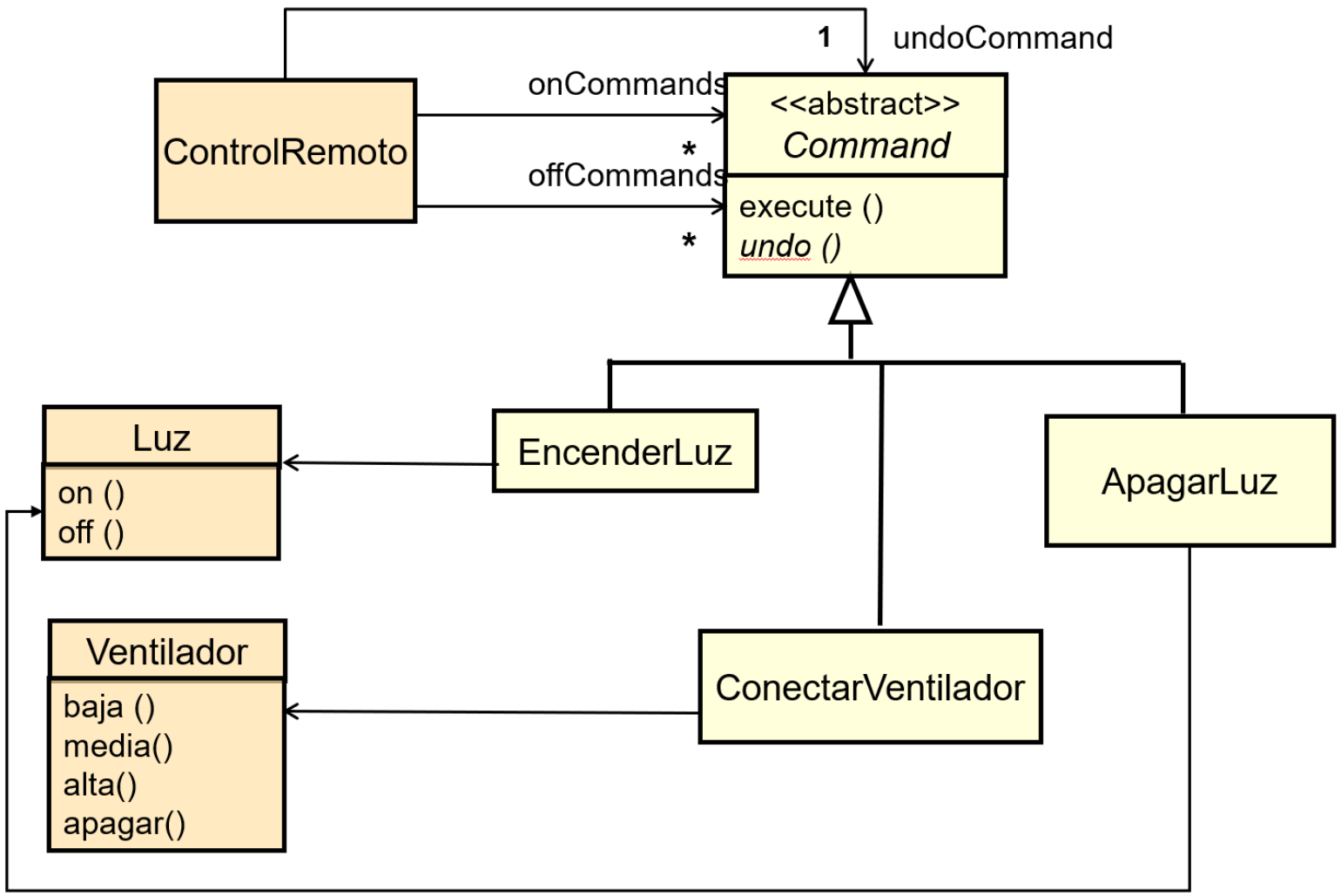


(extraído de “Head First Design Patterns”)

(extraído de “Head First Design Patterns”)

```
public class ControlRemoto {
    Command[]   onCommands;
    Command[]   offCommands;
    Command     undoCommand;
    public  ControlRemoto() {
          onCommands = new Command [7];
          offCommands = new Command [7];
          Command  noCommand = new NoCommand();
          for (int i = 0; i < 7; i++) {
              onCommands[i] = noCommand; offCommands[i] = noCommand;
          }
          undoCommand = noCommand
    }
    public void setCommand (int slot, Command onCommand, Command offCommand){
          onCommands[slot]  = onCommand; offCommands[slot] = offCommand;
    }
    public void botonOnPulsado (int slot) {
          onCommands[slot].execute();
          undoCommand = onCommands[slot];
    }
    public void botonOffPulsado (int Slot) {
          offCommands[slot].execute();
          undoCommand = offCommands[slot];
    }
    public void botonUndoPulsado () {
          undoCommand.undo();
    }
}
```

Command. Ejercicio “Mando de Control Remoto”




```
public class EncenderLuz {
   private Luz unaLuz;
   public EncenderLuz(Luz luz) {
        unaLuz = luz;
   }
   public void execute () {
        unaLuz.on();
   }
   public void undo () {
        unaLuz.off();
   }
}
```

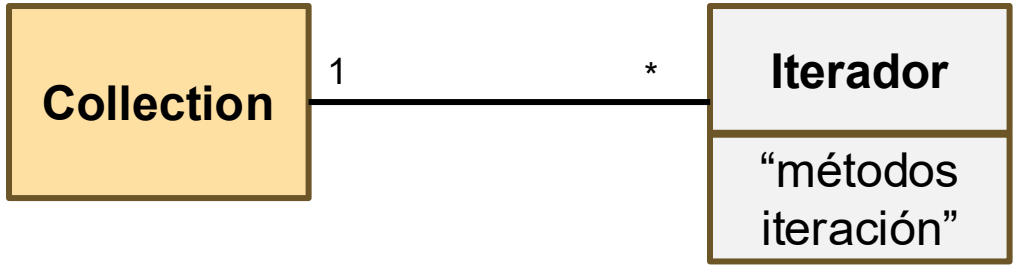
Iterador(Iterator)

Propósito

Proporciona una forma para acceder a los elementos de una estructura de datos sin exponer los detalles de la representación. Proporcionado por la mayoría de los lenguajes de programación

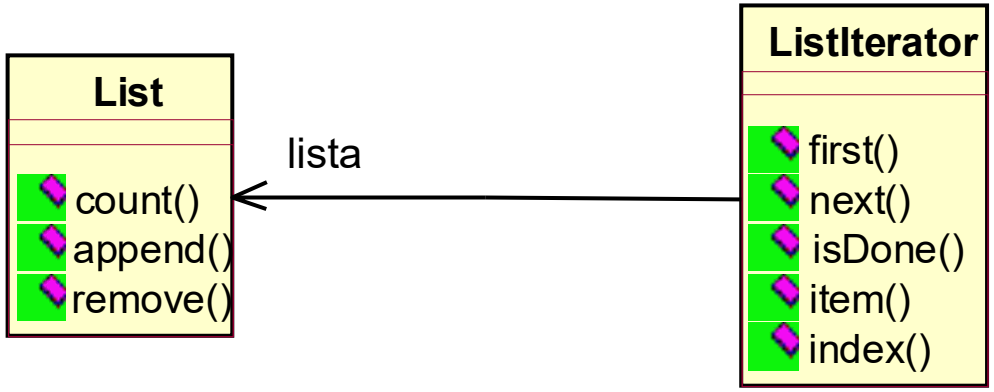
Motivación

- Un objeto contenedor (colección) debe permitir una forma de recorrer sus elementos **sin exponer su estructura interna,** permitiendo **diferentes métodos de recorrido** y recorridos **concurrentes**
- Las clases colección no implementan la iteración.
- Dos posibilidades:
  - **Externos**: recorrido **controlado por el código** que usa el iterador **(tarea del programador)**
  - **Internos**: recorrido **controlado por el propio iterador** (se libera al programador que sólo dice lo que quiere)



Iterador **Externo**: Ejemplo

No son clases de Java, ejemplo de DP, GoF

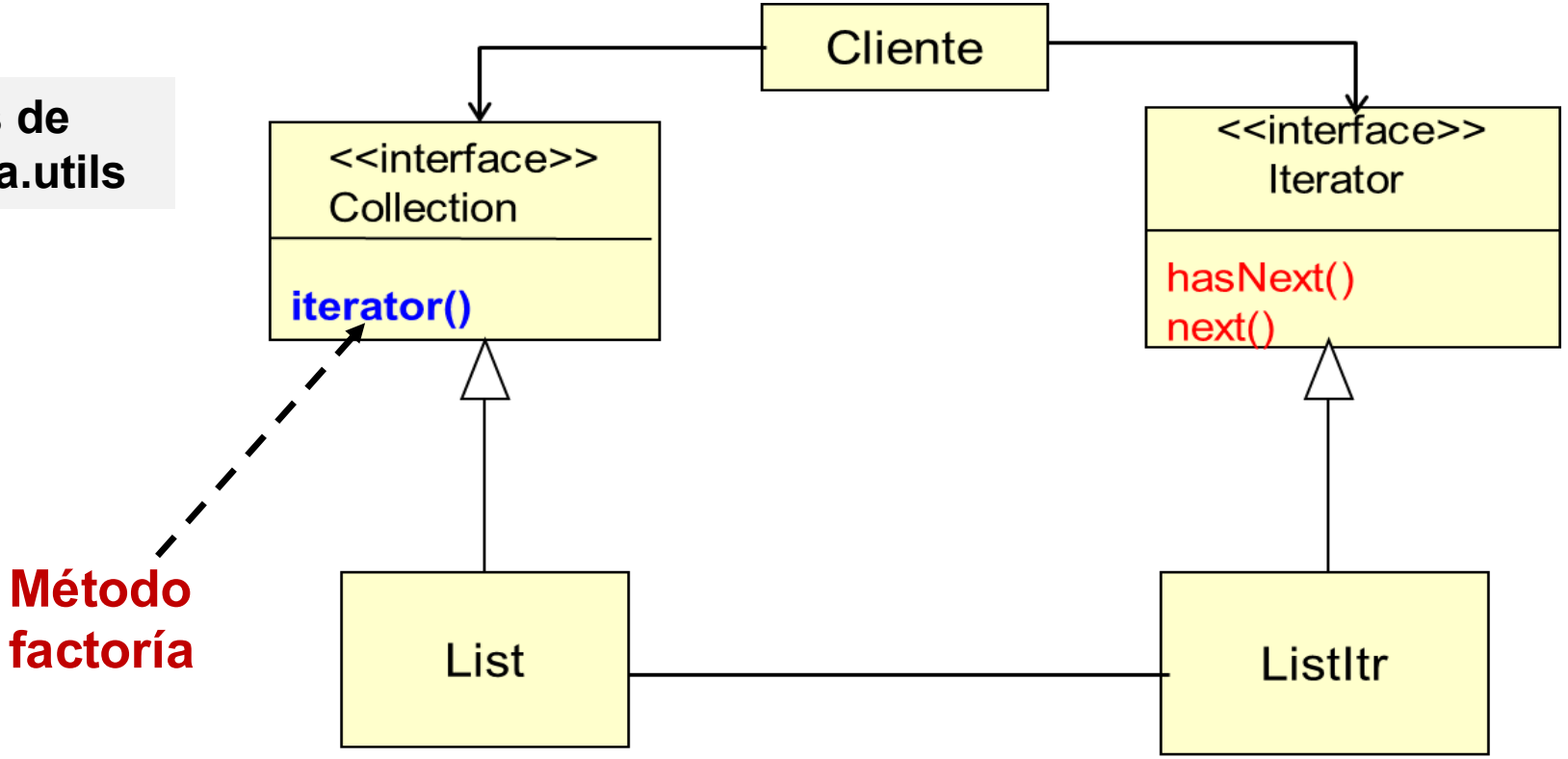



```
List lista = new List();
...
ListIterator iterator = new ListIterator(lista);
iterator.first();
while (!iterator.isDone()) {
   Object item = iterator.item();
   // código para procesar item
   iterator.next();
}
...
```

Necesario conocer qué iterador aplicar

Iterador Externo + Método Factoría (Java)

Son clases de librería java.utils




```
List<Cuenta> cuentas;
...
Iterator<Cuenta> it = cuentas.iterator();
while (it.hasNext()) {
     Cuenta cu = it.next();
      if (cu.getSaldo>100000) cu.reintegro(600)
}
```

Iterador Externo en Java < 5

```
interface Iterator<E> {
      boolean hasNext();
      E  next();
      void remove();
}

interface Collection<E>
     extends Iterable<E> {..
  Iterator<E> iterator();
  // retorna un iterador
 // sobre la colección
```

```
List<Cuenta> cuentas;

int num= 0;
Iterator<Cuenta> it =
        cuentas.iterator();
while (it.hasNext()) {
    Cuenta cu = it.next();
     if (cu.getSaldo()>10000)
                num = num+ 1;
}
```

Iterador Interno **forEach** en Java 5

```
List<Cuenta> cuentas;

int num = 0;
for (Cuenta cu: cuentas)
    if (cu.getSaldo()> 10000)
            num = num + 1;
```

Iterador **interno**

- El recorrido es controlado internamente, el programador es liberado.
- Permiten iterar sobre una colección y ejecutar acciones como “seleccionar todos los elementos que cumplen una condición” o “aplicar una operación sobre cada elemento”.
- Requieren el mecanismo llamado ***“bloque de código”, closure, expresiones lambda.***
- Comunes en lenguajes OO tipados dinámicamente como Smalltalk, Python, y Ruby.

Ejemplos en Smalltalk: Dada una colección de cuentas `colCuentas`

Seleccionar *todas las cuentas con saldo mayor que 1000*

```
colCuentas select: [:cu | cu getsaldo > 1000]
```

Aplicar la operación *`reintegro:` a todas las cuentas*

```
colCuentas do: [:cu | cu reintegro: 1000]
```

- Sin lambdas es poco práctico, se debe encapsular los “parámetros” como objetos (patrón *Estrategia*) o bien implementar esquemas algorítmicos con un *Método Plantilla*.

Patrones de operaciones con colecciones

```
for (T:e col1)
     if (condicion(e))
                    col2.add(e);
```

Select / Filter

```
for (T:e col1)
     col2.add(accion(e));
```

Map

```
for (T:e col1)
          accion(e);
```

Do / for each

```
existe = false;
for (T:e col1)
   if (condicion(e)) {
        existe = true;
        break;
   }
```

Any?

Sorting, y más

Iteradores internos en clase `Stream`

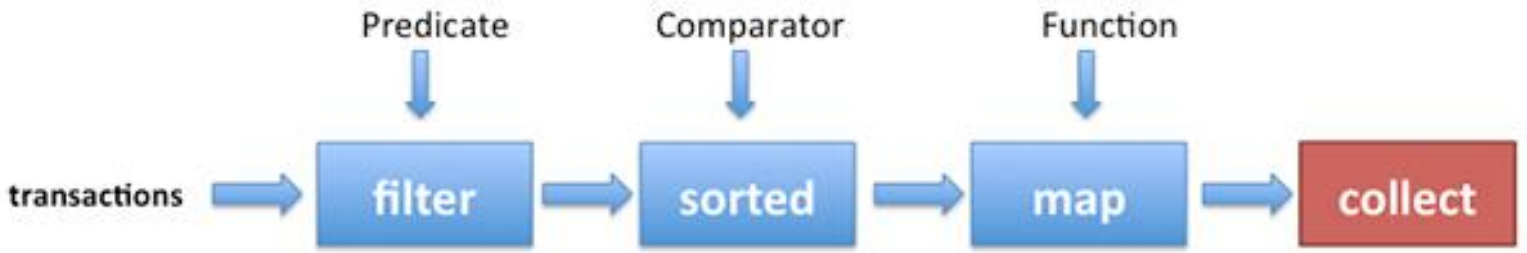
- Un *stream* (`java.util.Stream`) es una secuencia de elementos que soporta operaciones secuenciales y paralelas, entre ellas iteradores internos.
- Las colecciones incluyen dos métodos para obtener `streams` (`stream` y `parallelStream`) y la clase `Stream` ofrece métodos para iterar sobre su elementos (*iteradores internos*)

Ejemplo: contar las cuentas con un saldo positivo en una Colección `cuentas` de objetos `Cuenta`.

```
long contador = cuentas.stream()
        .filter(cu -> cu.getSaldo() > 0)
        .count();
```

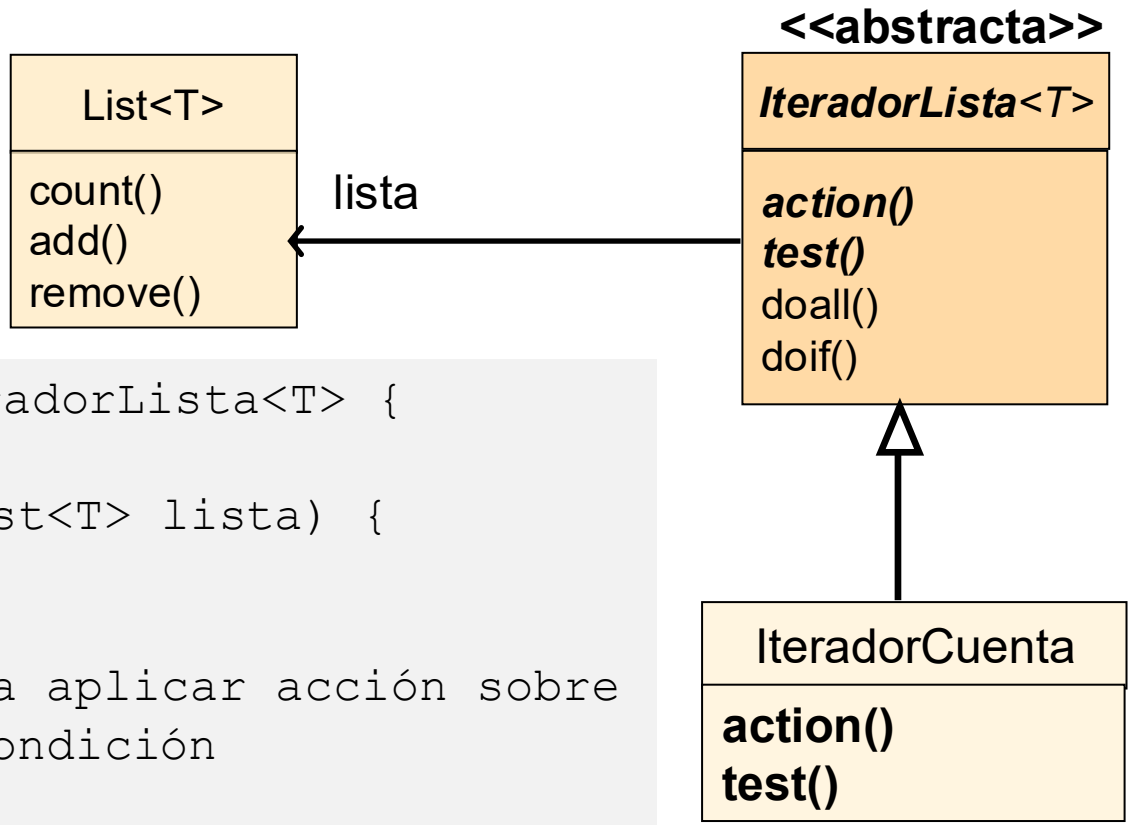
Streams (estudiados en el tema 3)

"Obtener lista de identificadores de transacciones de cierto tipo ordenadas por el valor de la transacción de mayor a menor."




```
List<Integer> transactionsIds = transactions.stream()
        .filter(t -> t.getType() == Transaction.COMPRADIVISAS)
        .sorted(comparing(Transaction::getValue).reversed())
        .map(Transaction::getId)
        .collect(toList());
```

Iterador Interno con **Método Plantilla**




```
public abstract class IteradorLista<T> {
   private List<T> lista;
   public IteradorLista(List<T> lista) {
        this.lista = lista;
   }
   // iterador interno para aplicar acción sobre
   //objetos que cumplan condición
   public void doIf () {
        for (T o : lista)
            if (test(o)) action(o);
         }
   }
   protected abstract boolean test (T element);
   protected abstract void action (T element);
}
```

Ejemplo Iterador Interno con Método Plantilla

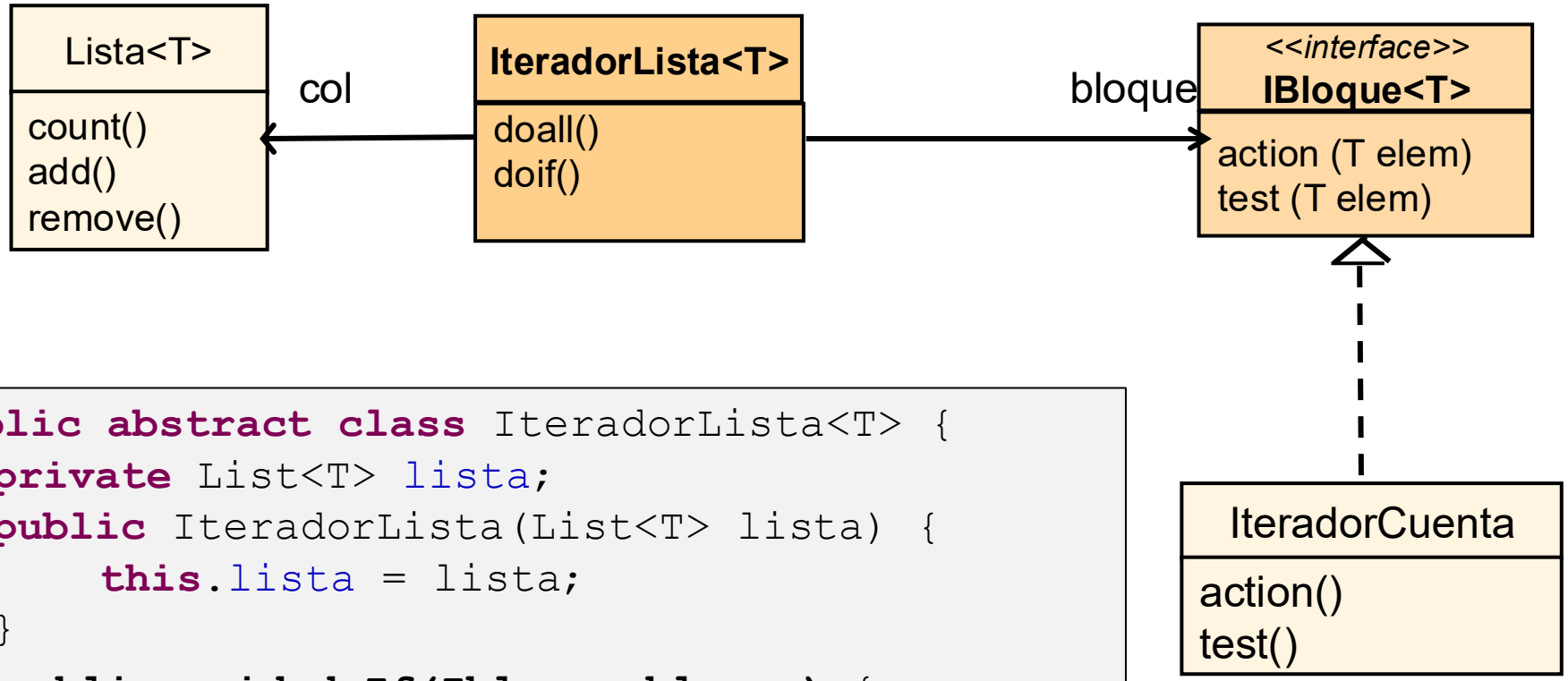
```
// imprimir cuentas cuyo saldo supere cierta cantidad
class IteradorSaldoCuenta extends IteradorLista<Cuenta> {
     int cantidad;
     IteradorSaldoCuenta (List<Cuenta> cc, int cant){
            super(cc);
            cantidad = cant;
     }
     protected boolean test (Cuenta elem){
        return (elem.getSaldo()>cantidad);
     }
    protected void action (Cuenta elem){
        System.out.println(elem.getCodigo()+ ” ” + elem.getSaldo());
    }
```

Ejemplo Iterador Interno

```
public class PruebaIterador {
    public class IteradorSaldoCuenta extends IteradorLista<Cuenta> {..}
    public static void main(String[] args) {
            PruebaIterador p = new PruebaIterador();
            ArrayList<Cuenta> lista = new ArrayList<Cuenta>();
            Cuenta c1 = new Cuenta (100, "123", "JGM");
            ...
            Cuenta c6 = new Cuenta (1000, "222", "MOGM");
            lista.add(c1);..; lista.add(c6);

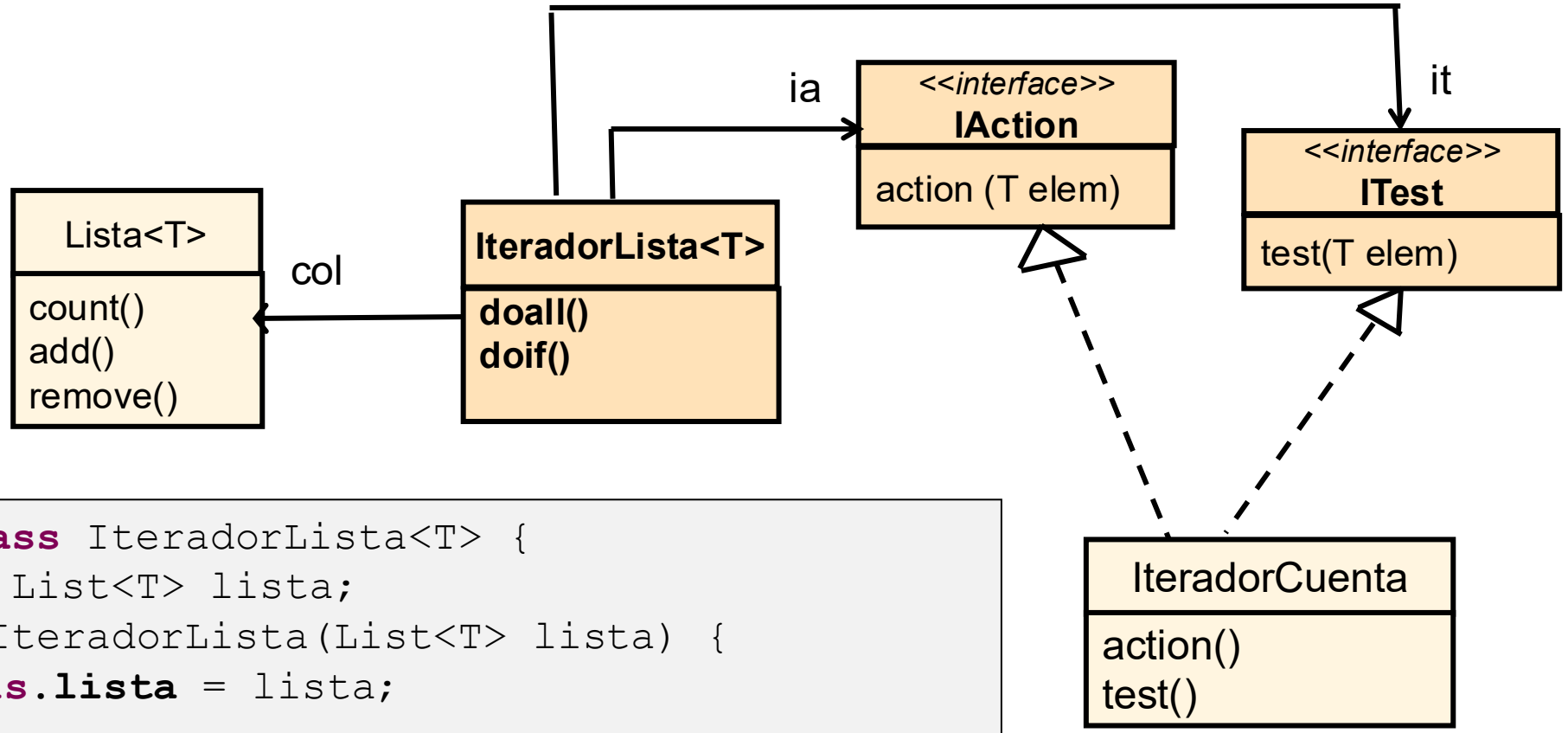
            p.new IteradorSaldo(lista,100).doIf();
    }
}
```

Iterador interno basado en composición (i)




```
public abstract class IteradorLista<T> {
   private List<T> lista;
   public IteradorLista(List<T> lista) {
        this.lista = lista;
   }
    public void doIf(Ibloque bloque) {
        for (T o : col)
           if (bloque.test(o))bloque.action(o);
    }
}
```

Iterador interno basado en composición (ii)




```
public class IteradorLista<T> {
  private List<T> lista;
  public IteradorLista(List<T> lista) {
       this.lista = lista;
  }
  public void doIf (ITest<T> it, Iaction<T> ia) {
       for (T o : lista)
            if (it.test(o)) ia.accion(o);
  }
}
```

```
public class PruebaIterador {
    private int valor;
    private ArrayList<Cuenta> lista;
    public PruebaIterador() {..
       lista = new ArrayList<Cuenta>();
       Cuenta c1 = new Cuenta (100, "123", "JGM");
       ..
       Cuenta c6 = new Cuenta (1000, "222", "MOGM");
       lista.add(c1); ... ;lista.add(c6);
     }
     public int getValor() { return valor;}
     public void setValor(int valor) { this.valor = valor;}
     public void ejecutar() {
         IteradorLista<Cuenta> it = new IteradorLista<Cuenta>(lista);
         //clase interna anónima
         it.doIf(new IBloque<Cuenta>() {
             @Override
             public boolean test(Cuenta cuenta) {
                  return (cuenta.getSaldo()> valor);
             }
             @Override
             public void action(Cuenta cuenta) {
                  System.out.println(cuenta.getNombreTitular()+" "+cuenta.getSaldo());
             }
         }) ;
     }
     public static void main(String[] args) {
          PruebaIterador p = new PruebaIterador();
          p.setValor(Integer.parseInt(args[0]));
          p.ejecutar();
     }
}
```

Una clase con iteradores internos basada en las interfaces funcionales Predicate y Consumer (los nombres de los iteradores no coinciden con los de Java 8.

```
public class Iterador<T> {
   ArrayList<T> col;
   public Iterador(ArrayList<T> c) {
         col = c;
   }
   public ArrayList<T> select(Predicate<T> p){
         ArrayList<T> result = new ArrayList<T>();
         for(T o: col)
               if (p.test(o)) result.add(o);
         return result;
   }
   public ArrayList<T> reject(Predicate<T> p){
         ArrayList<T> result = new ArrayList<T>();
         for(T o: col)
              if (!p.test(o)) result.add(o);
         return result;
  }
  public void doIf(Predicate<T> p, Consumer<T> c){
         for (T o: col)
              if (p.test(o)) c.accept(o);
  }
  public void forEach(Consumer<T> c){
         for (T o: col)
               c.accept(o);
  }
}
```

Iterador

Consecuencias

- Simplifica la interfaz de una colección al extraer los métodos de recorrido
- Permite varios recorridos concurrentes
- Soporta variantes en las técnicas de recorrido

Implementación

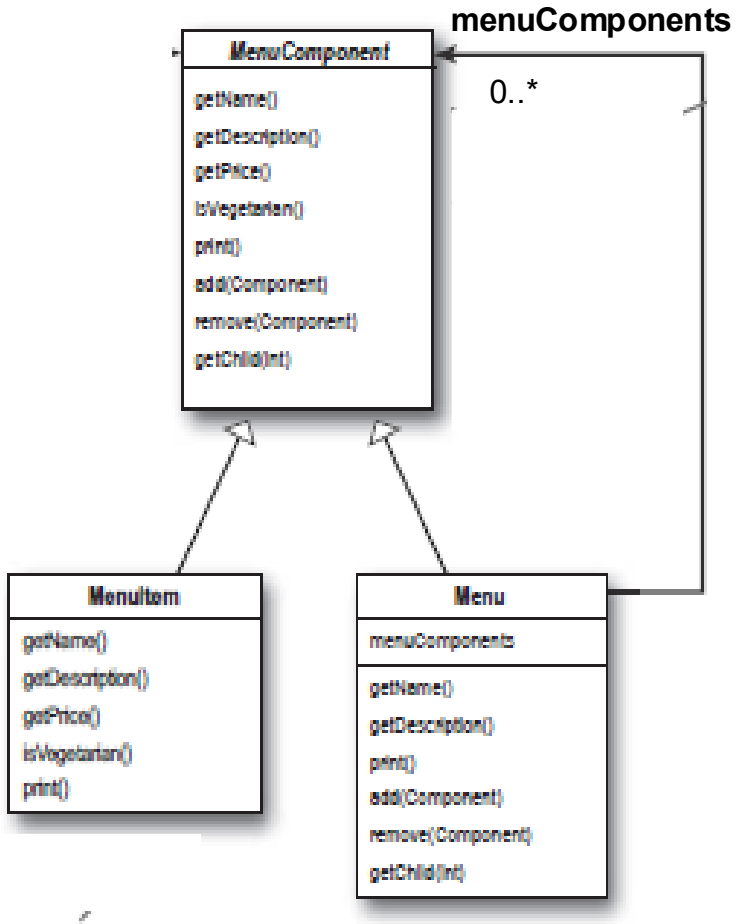
- ¿Es posible **modificar** la colección durante la iteración? -> Iterador Robusto
- Colección e iterador son clases íntimamente relacionadas: clases amigas en C++ y **clases internas en Java** para iteradores externos.
- Puede ser usado junto con el **patrón *Composite*.**

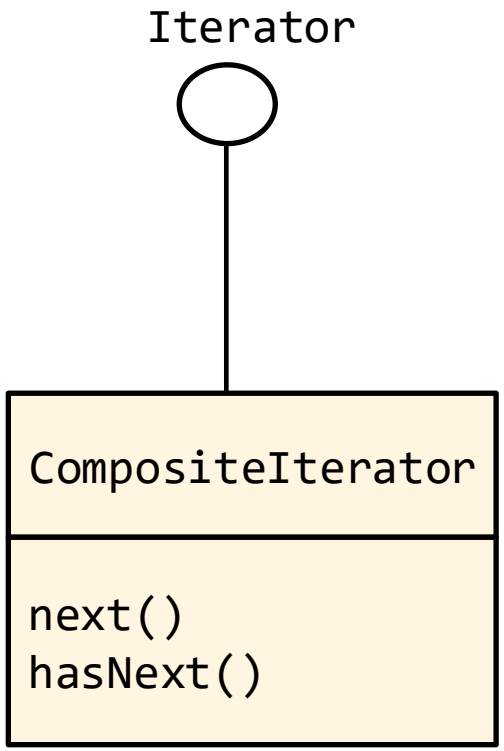
```
public interface ListIterator<E> extends Iterator<E>
```

Methods

Modifier and Type	Method and Description
void	**add(E e)** Inserts the specified element into the list (optional operation).
boolean	**hasNext()** Returns `true` if this list iterator has more elements when traversing the list in the forward direction.
boolean	**hasPrevious()** Returns `true` if this list iterator has more elements when traversing the list in the reverse direction.
E	**next()** Returns the next element in the list and advances the cursor position.
int	**nextIndex()** Returns the index of the element that would be returned by a subsequent call to **next()**.
E	**previous()** Returns the previous element in the list and moves the cursor position backwards.
int	**previousIndex()** Returns the index of the element that would be returned by a subsequent call to **previous()**.
void	**remove()** Removes from the list the last element that was returned by **next()** or **previous()** (optional operation).
void	**set(E e)** Replaces the last element returned by **next()** or **previous()** with the specified element (optional operation).

Composite + Iterador
(extraído de “Head First Design Patterns”)





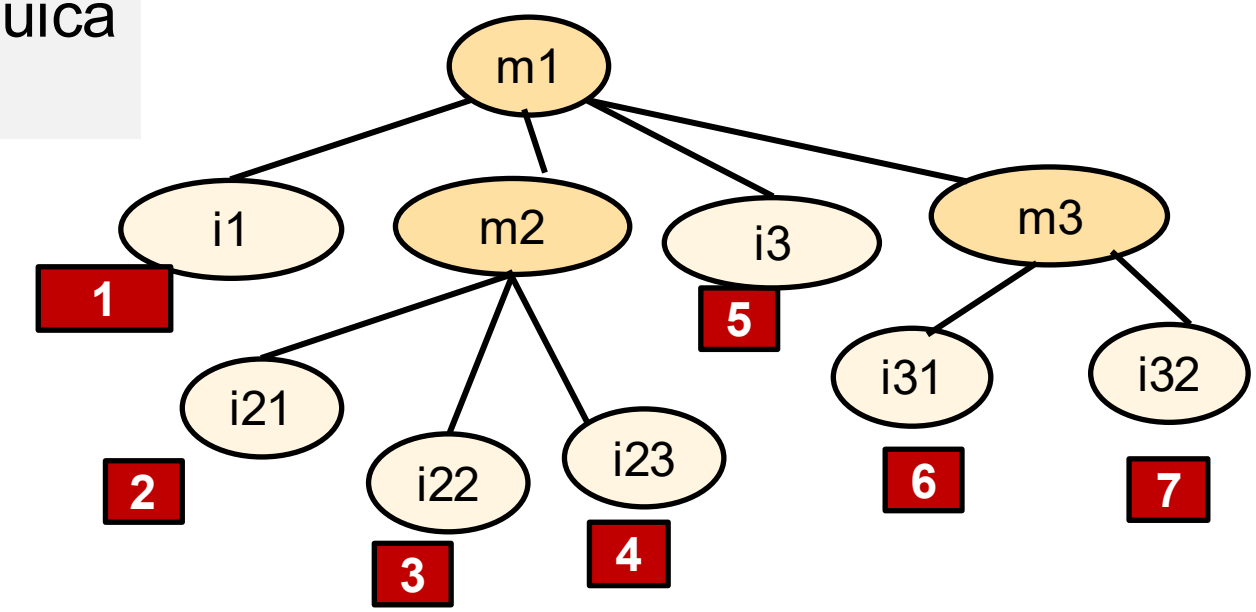

```
void print(){
    for (MenuComponent mc: menuComponents)
            mc.print()
}
```

Se aplica una operación sobre todos los elementos

Se desea visitar cada element de un composite

Composite + Iterador

Estructura jerárquica de un menú



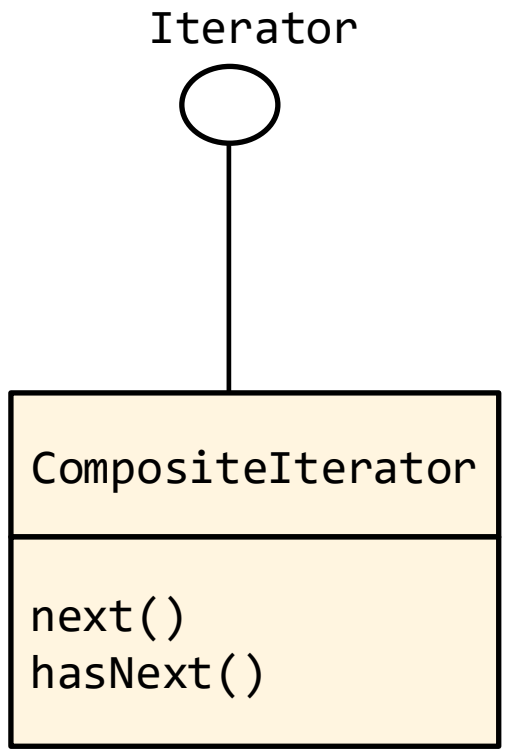

```
// se recorren los elementos aplicando una operación
void print(){
    for (MenuComponent mc: menuComponents)
            mc.print()
}
```

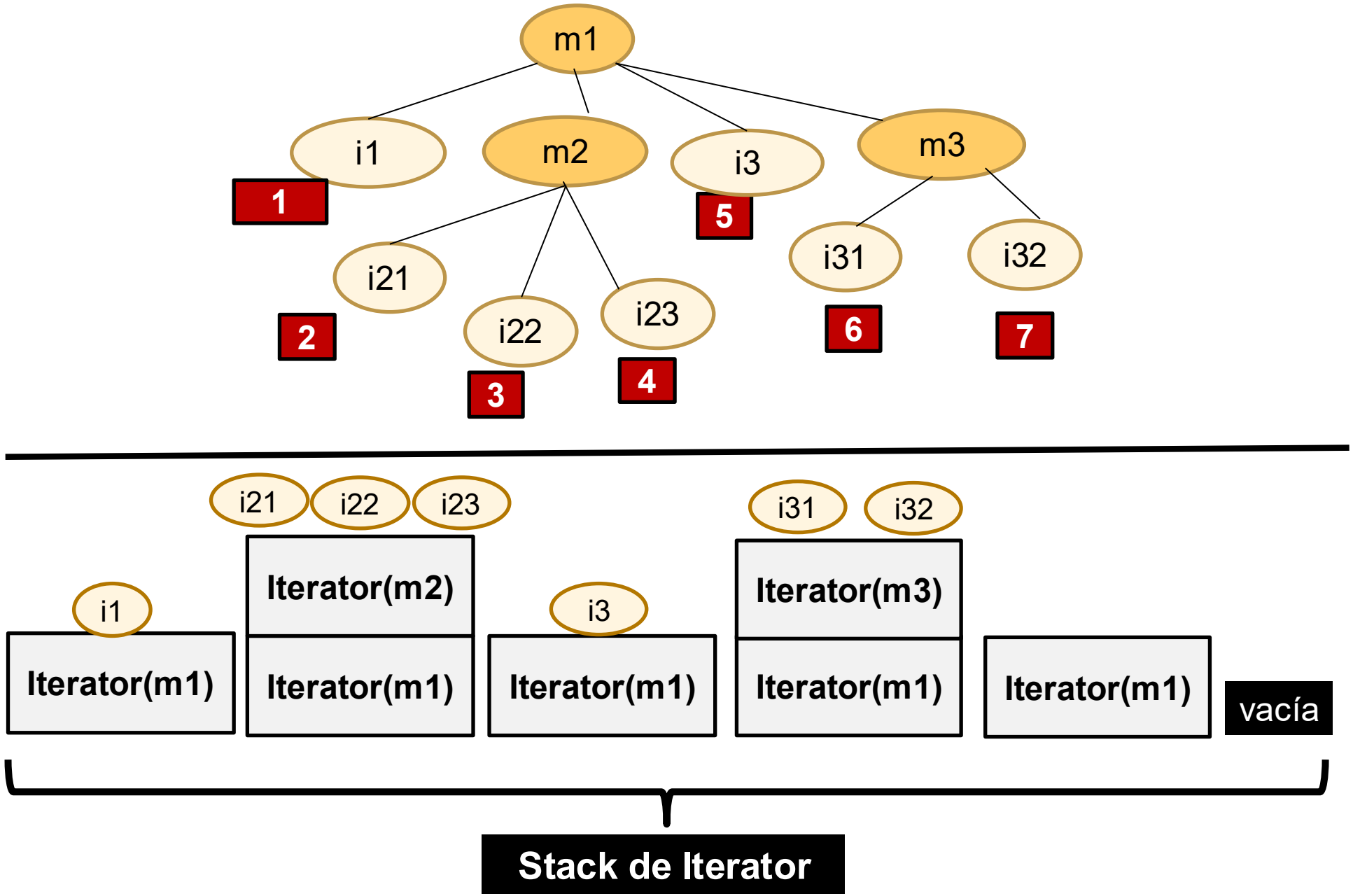
Composite + Iterador

¿Cómo iterar externamente un composite para visitar sus elementos?

```
abstract class MenuComponent {..
    public Iterator createIterator();
```

```
class Menu extends MenuComponent {..
    private ArrayList<MenuComponent> menuComponents;
    private Iterator iterator = null;
    public Iterator createIterator(){
        if (iterator==null){
            iterator = new CompositeIterator(
                                menuComponents.iterator());
        }
        return iterator;
    }
```




m1
i1
m2
i3
m3
1
5
i21
i31
i32
2
i22
i23
6
7
3
4
i21
i22
i23
i31
i32
i1
Iterator(m2)
i3
Iterator(m3)
Iterator(m1)
Iterator(m1)
Iterator(m1)
Iterator(m1)
Iterator(m1)
vacía
Stack de Iterator

```
public class CompositeIterator implements Iterator {
    Stack stack = new Stack();

    public CompositeIterator(Iterator iterator) {
        stack.push(iterator);
    }

    public Object next() {
        if (hasNext()) {
            Iterator iterator = (Iterator) stack.peek();
            MenuComponent component = (MenuComponent) iterator.next();
            if (component instanceof Menu) {
                stack.push(component.createIterator());
            }
            return component;
        } else {
            return null;
        }
    }

    public boolean hasNext() {
        if (stack.empty()) {
            return false;
        } else {
            Iterator iterator = (Iterator) stack.peek();
            if (!iterator.hasNext()) {
                stack.pop();
                return hasNext();
            } else {
                return true;
            }
        }
    }

    public void remove() {
        throw new UnsupportedOperationException();
    }
}
```

The iterator of the top level composite we're going to iterate over is passed in. We throw that in a stack data structure.

Okay, when the client wants to get the next element we first make sure there is one by calling hasNext()...

If there is a next element, we get the current iterator off the stack and get its next element.

If that element is a menu, we have another composite that needs to be included in the iteration, so we throw it on the stack. In either case, we return the component.

To see if there is a next element, we check to see if the stack is empty; if so, there isn't.

Otherwise, we get the iterator off the top of the stack and see if it has a next element. If it doesn't we pop it off the stack and call hasNext() recursively.

Otherwise there is a next element and we return true.

We're not supporting remove, just traversal.

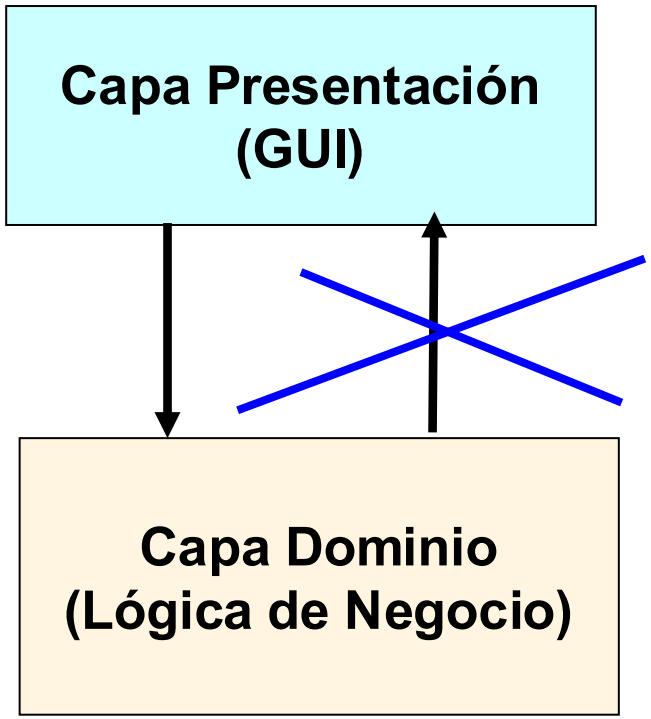
Patrón Observer / Publish-Subscribe

Propósito

Define una dependencia uno-a-muchos entre objetos, de modo que **cuando cambia el estado de un objeto**, **todos sus dependientes** son automáticamente notificados.

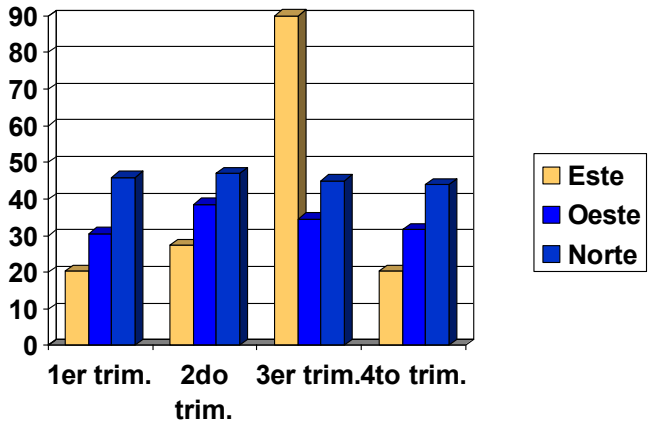
Motivación

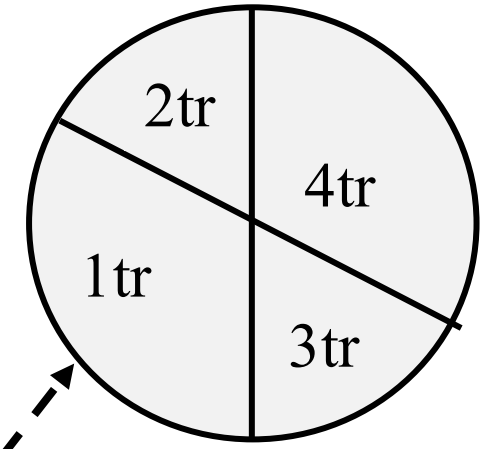
Ejemplo: **Separación *Modelo-Vista***



Los objetos del dominio **no deben conocer** a los objetos de la interfaz, **¿cómo les notifican de un cambio?**

Observer: Actualizar varias vistas cuando cambia el modelo



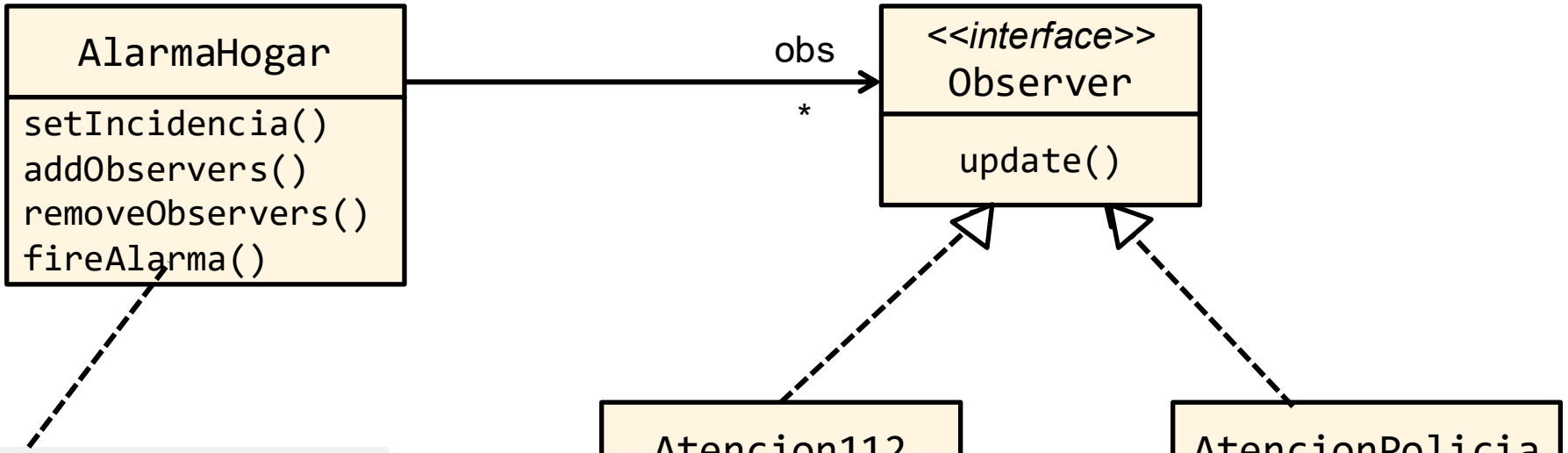


Modelo

t1=20
t2=25
t3=90
t4=20

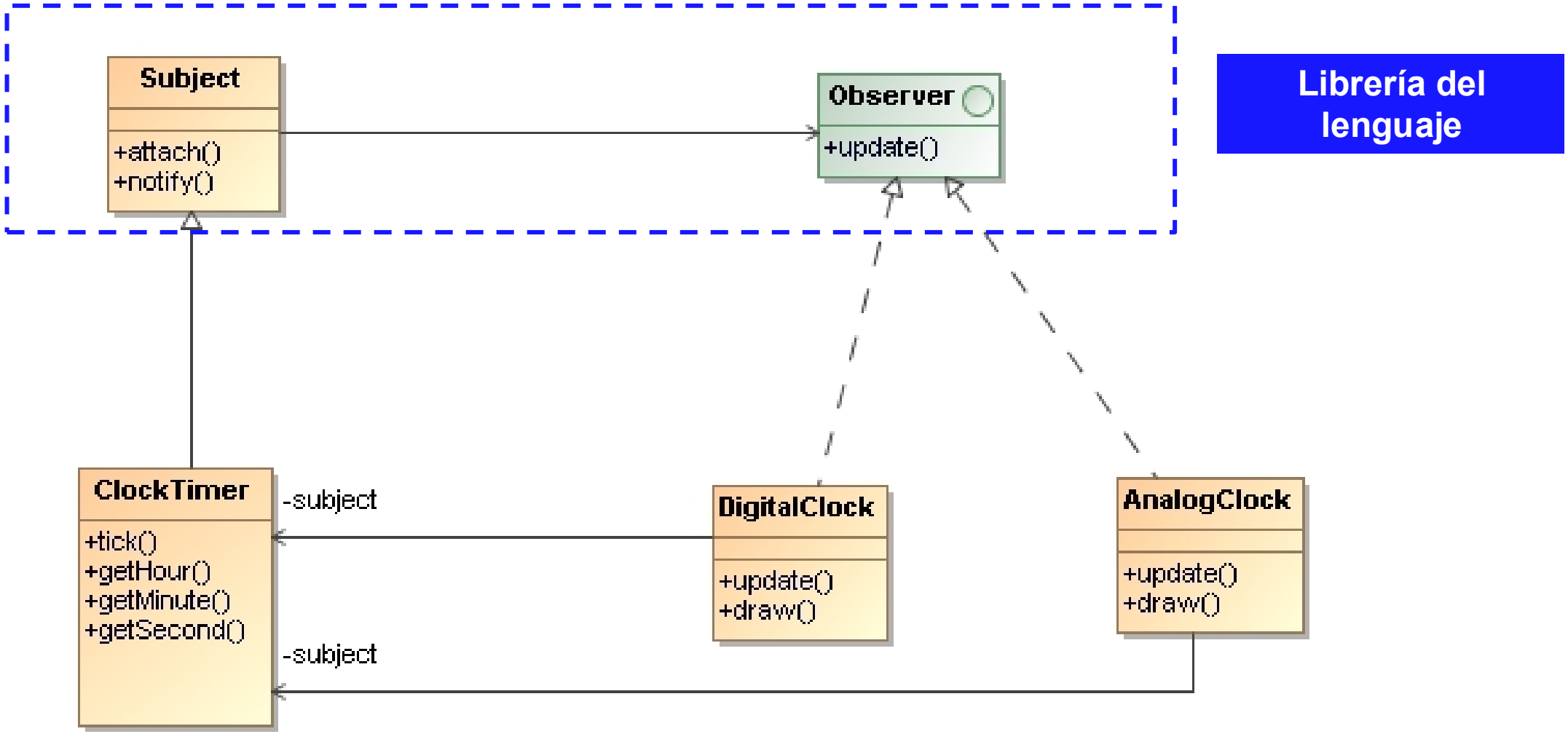
	1tr.	2tr.	3tr.	4tr.
Este	20	25	90	20
Oeste	30	38	32	32
Norte	47	47	45	45

Observer: una implementación simple ad-hoc

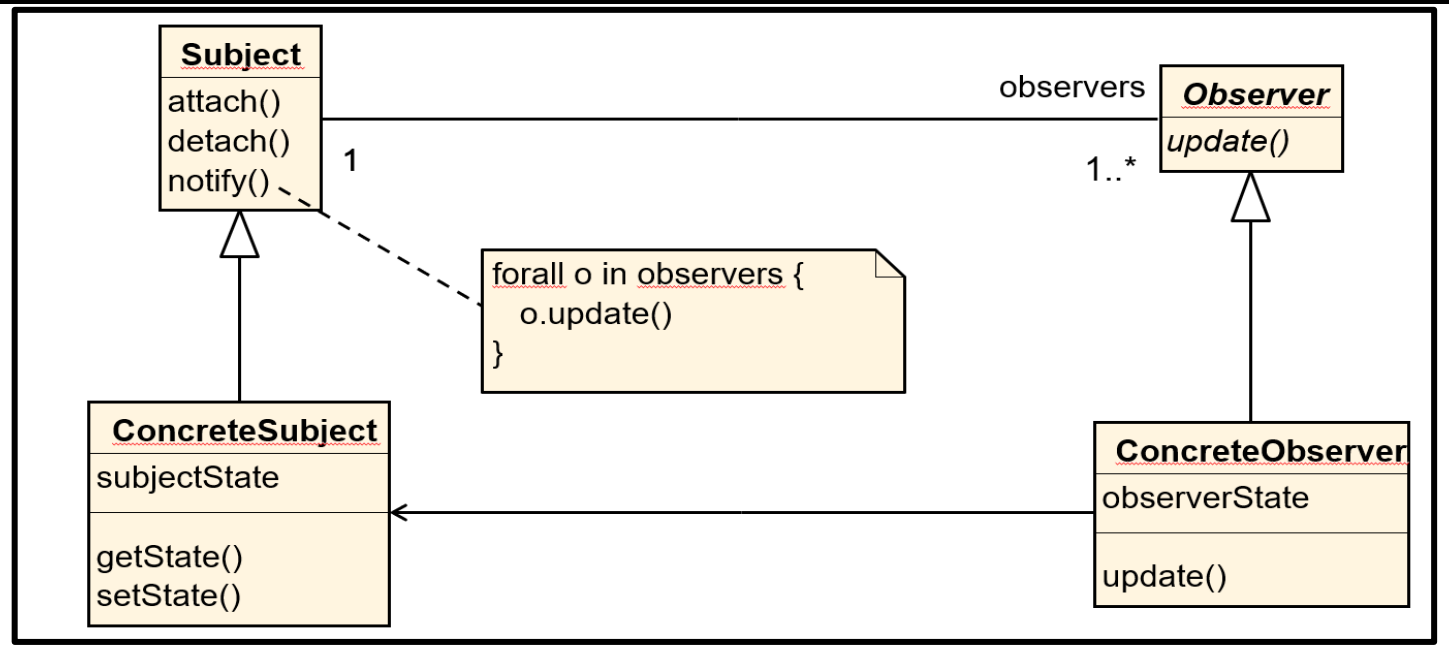


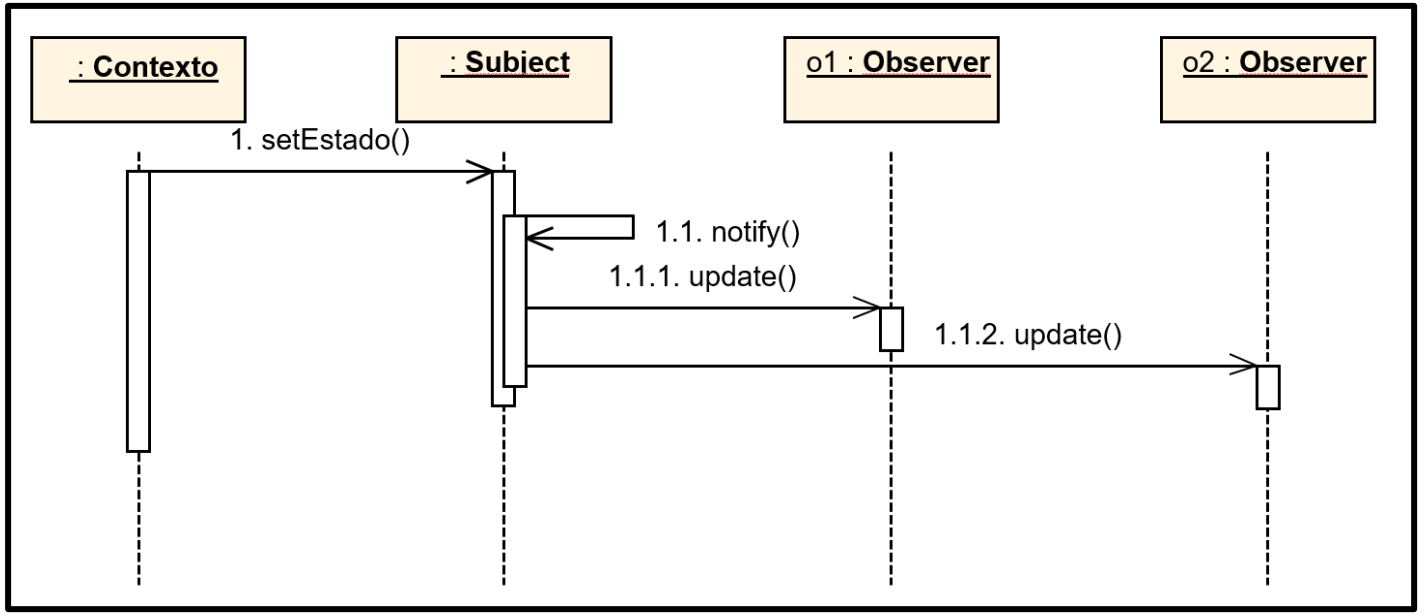

```
void fireAlarma(){
   obs.foreach(o ->
           o.update(this);
}
```

Observer. Ejemplo “Alarma” (DP, GoF)



Observer: Estructura y Comportamiento GoF





Observer: Aplicabilidad

- Cuando un cambio de estado en un objeto requiere cambios en otros objetos, y no sabe sobre qué objetos debe aplicarlos.
- Cuando un objeto debe ser capaz de notificar algo (p.e., un evento) a otros objetos, sin hacer asunciones sobre quiénes son estos objetos.

Observer: Consecuencias

- Acoplamiento abstracto y mínimo entre Subject y Observer:
  - Puedo **reutilizarlos por separado.**
  - Pueden estar en **diferentes capas.**
  - Pueden añadirse nuevos tipos de *observers* sin modificar el *subject.*
  - *ConcreteSubject* no necesita conocer las clases concretas de *Observers.*
- Es posible añadir y eliminar objetos observers en cualquier instante.

Observer: Implementación

- Un observer puede estar ligado a más de un subject: la operación `update` tendrá como argumento el subject.
- Un subject puede notificar varios eventos: un *observer* debe indicar el evento de interés al registrarse.
- **¿Quién dispara la notificación?** Normalmente, **métodos `setters`** en la clase `Subject`, en vez de clases clientes de la clase `Subject`.
- **¿Cuánta información sobre el cambio** se le pasa a los observers con la notificación? Normalmente, el subject y/o un objeto con la información.

Observer en Java

Modelo de Delegación de Eventos es normalmente usado. Inicialmente una implementación básica con la clase `Observable` y la interfaz `Observer`.

```
public class Observable {
    private boolean changed = false;
    private Vector obs;
    public Observable() {
          obs = new Vector();
    }
     public synchronized void addObserver (Observer o) {..}
     public synchronized void deleteObserver (Observer o) {..}
     public void notifyObservers (Object arg) {..}
     protected synchronized void setChanged() {..}
     protected synchronized void clearChanged() {..}
     public synchronized boolean hasChanged() {.. }
```

Observer en Java

Interfaz funcional

```
public interface Observer {
      void update (Observable o, Object arg);
}
```

Argumento recibido del método notifyObservers, puede indicar el tipo de cambio

```
public void notifyObservers(Object arg) {
        Object[] arrLocal;
         synchronized (this) {
               if (!changed)
                 return;
             arrLocal = obs.toArray();
             clearChanged();
         }

         for (int i = arrLocal.length-1; i>=0; i--)
             ((Observer)arrLocal[i]).update(this, arg);
}
```

Observer en Java. Ejemplo

```
public class Producto extends Observable {
   private String nombre;
   private float precio;
   public Producto(String nombre, float precio) {
               this.nombre = nombre;
               this.precio = precio;
   }
   //aquí métodos getters
   public void setNombre(String nombre) {
               this.nombre = nombre;
               setChanged();
               notifyObservers(nombre);
   }
   public void setPrecio(float precio) {
               this.precio = precio;
               setChanged();
               notifyObservers(new Float(precio));
   }
}
```

Observer en Java. Ejemplo

```
public class PrecioObserver implements Observer {
   private float precio;
   public PrecioObserver() {
               this.precio= 0;
   }
  public void update(Observable obj, Object arg) {
       if (arg instanceof Float) {
           precio = ((Float)arg).floatValue();
           System.out.println(
               “PrecioObserver: Precio ha cambiado a " + precio);
       } else {
           System.out.println(
                “PrecioObserver: Algo diferente ha cambiado!");
    }
  }
}
```

Observers pueden ser implementados con expresiones lambda
(ejemplo de Java 8 Lambdas, Richard Warburton)

```
public interface LandingObserver {
	public void observeLanding(String name);
}
```

```
public class Moon {
	private final List<LandingObserver> observers =
                                        new ArrayList<>();

	public void land(String name) {
		for (LandingObserver observer : observers) {
			observer.observeLanding(name);
		}
	}
	public void addLandingObserver(LandingObserver obs) {
		observers.add(obs);
	}
}
```

Observers pueden ser implementados con expresiones lambda (ejemplo de Java 8 Lambdas, Richard Warburton)

```
Moon moon = new Moon();

moon.addLandingObserver(name ->
          if (name.contains("Apollo"))
                System.out.println("We made it!");
);

moon.addLandingObserver(name -> {
      if (name.contains("Apollo"))
            System.out.println("They're distracted,
                                lets invade earth!");
});

moon.land("An asteroid");
moon.land("Apollo 11");
```

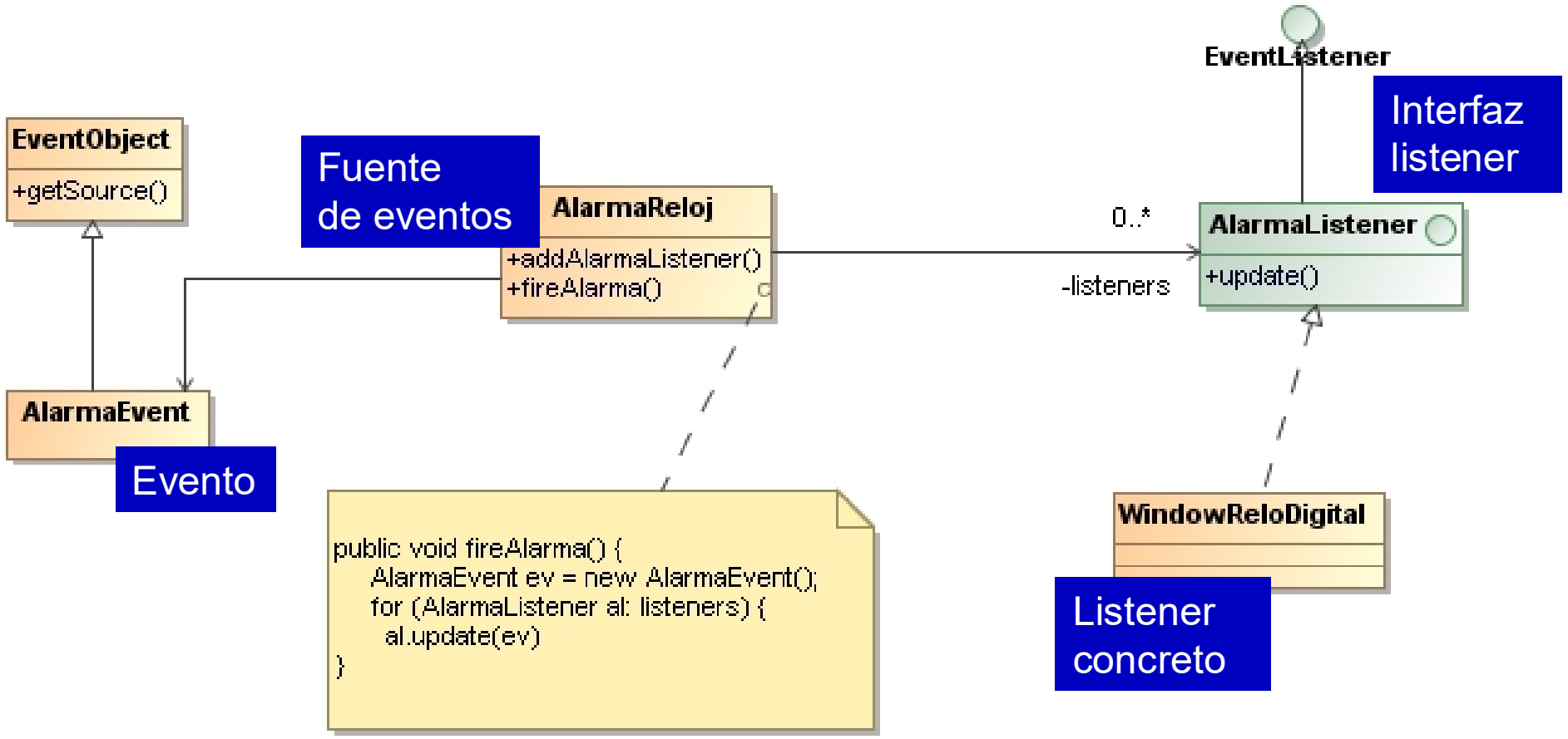
Modelo de delegación de eventos (MDE)

- Java 1.1 introdujo un nuevo modelo de notificación de eventos basado en el patrón *Observer*.
- Objetos (p.e., un widget Swing) que pueden generar eventos son llamados *fuentes de eventos*.
- Objetos que desean ser notificados de eventos (p.e. procedentes de una GUI) son llamados *oyentes de eventos* (*listeners*).
- Los *listeners* deben registrarse en las *fuentes e* implementan una interfaz con los métodos que deben ser llamados por la *fuente* cuando ocurre el evento.



Una *fuente* juega el papel de *ConcreteSubject* y un *listener* el de *ConcreteObserver*.

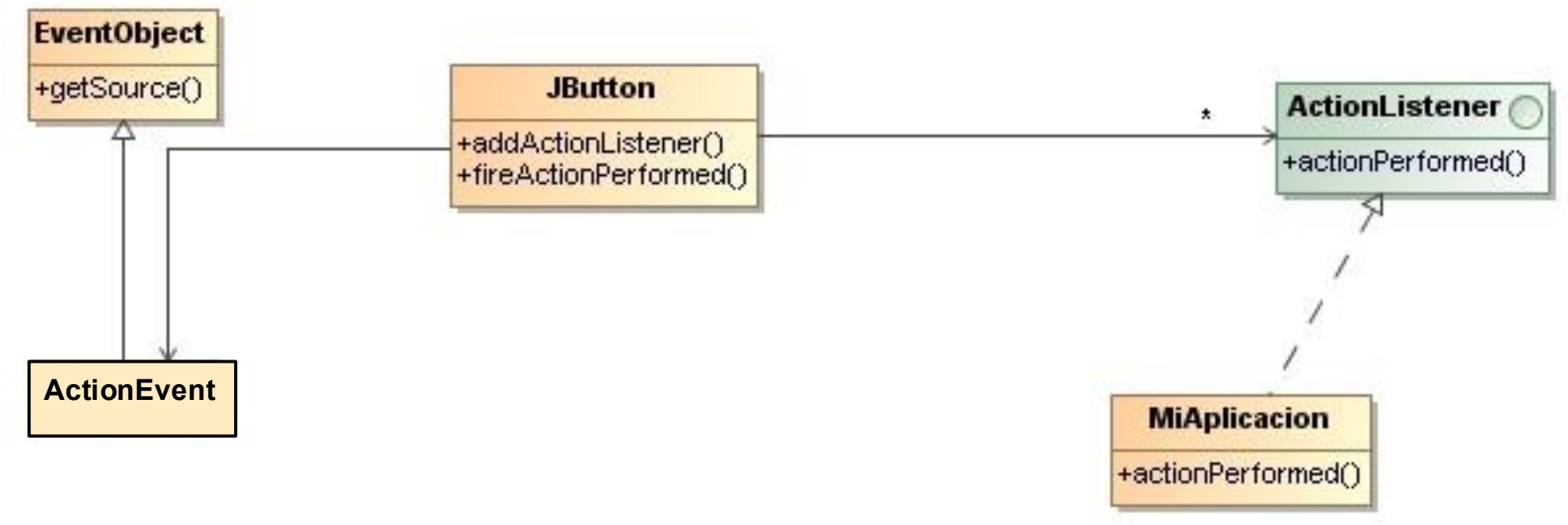
MDE Java. Ejemplo



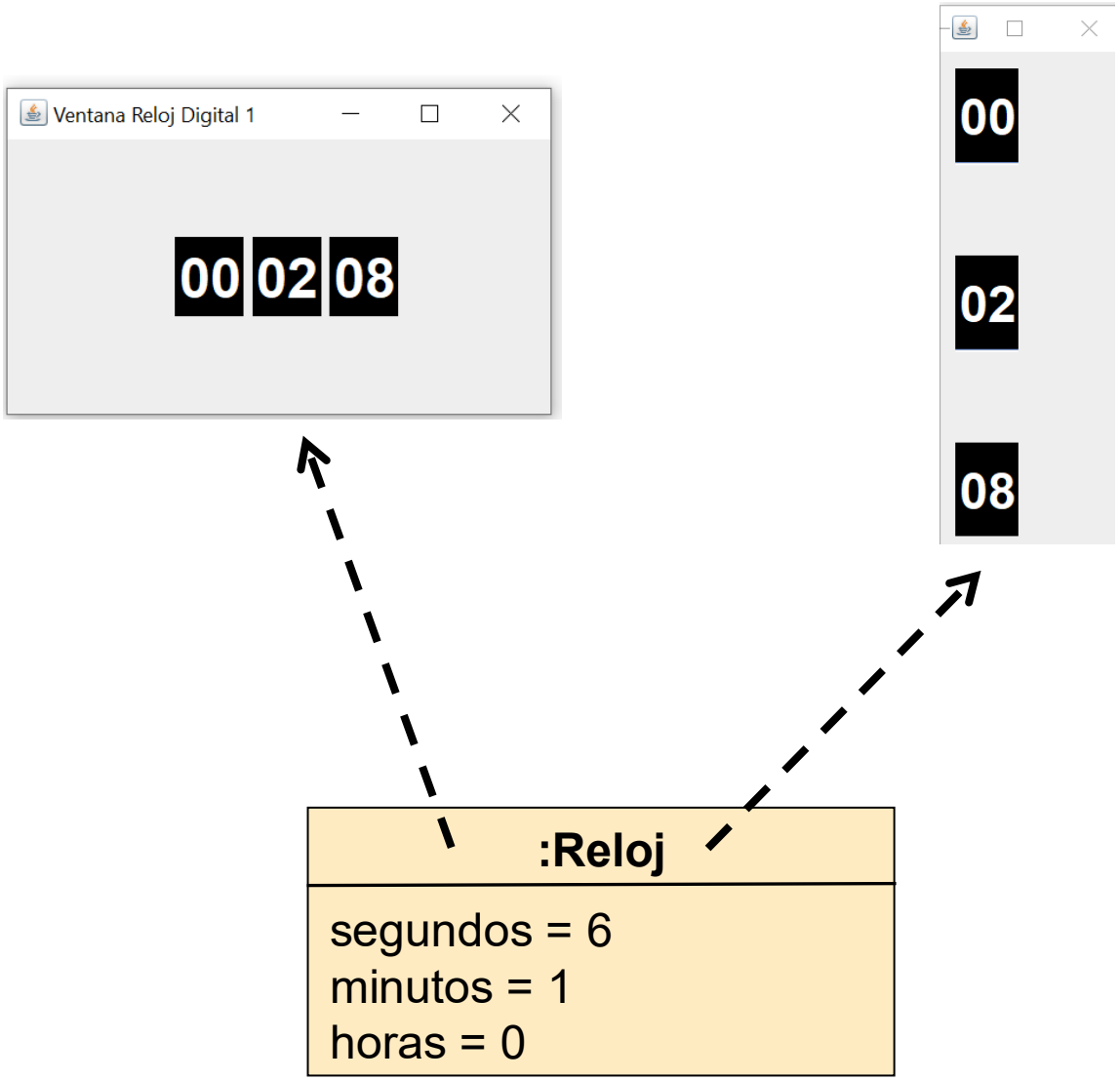
Los listeners pueden ser implementados con expresiones lambda

MDE: Ejemplo en Swing

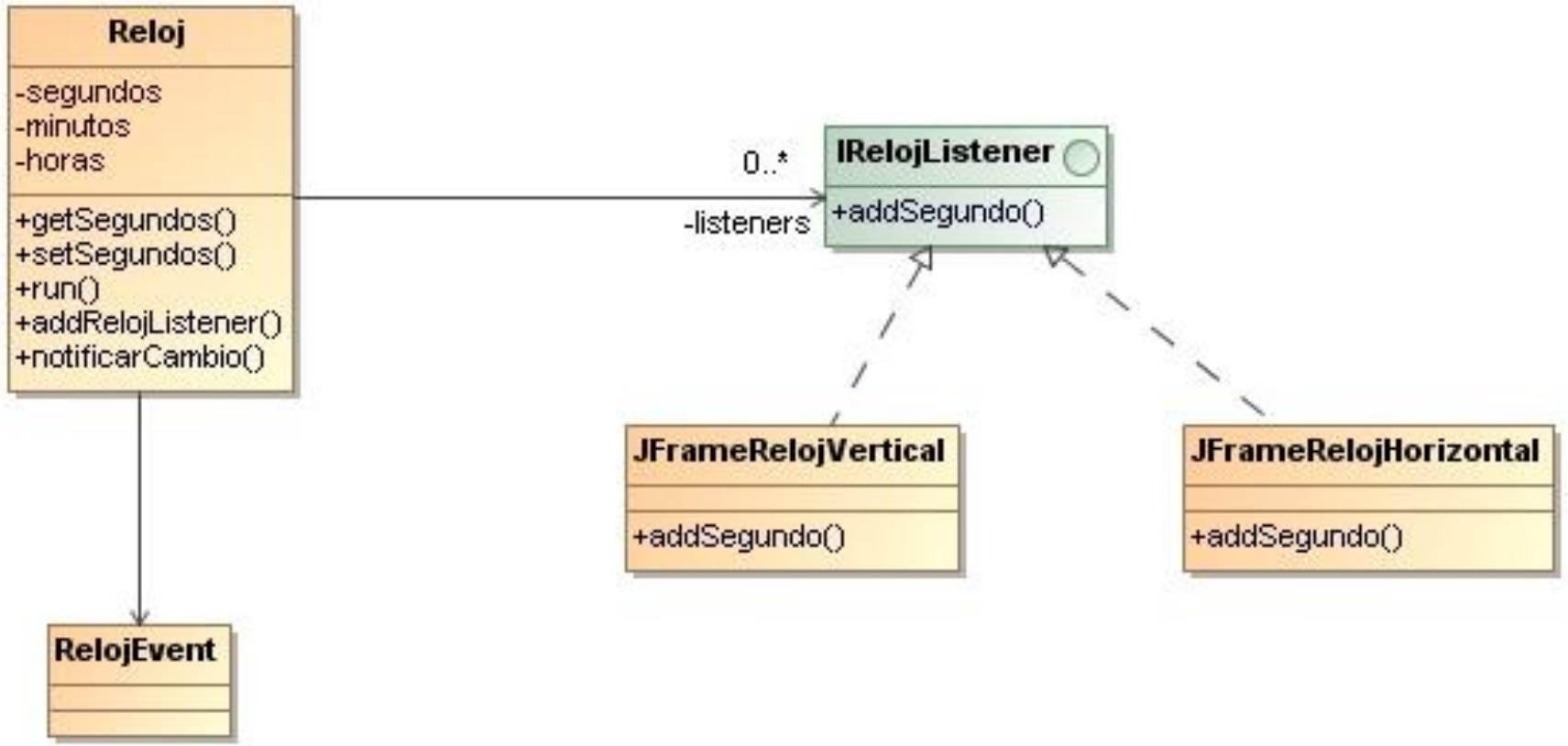
- **`ActionEvent`** es el evento producido
  - Ejemplos: Clic en botón, pulsar «enter» en un campo de entrada
- **`ActionListener`** es la interfaz listener:
  - `public void` **`actionPerformed`**`(ActionEvent e)`
- Método para añadir un oyente de ese evento en la fuente
  - `public` **`addActionListener`**`(ActionListener al)`



MDE: Ejemplo “actualizar varias ventanas”



MDE: Ejemplo “actualizar varias ventanas”



MDE: Ejemplo “actualizar varias ventanas”

```
public class RelojEvent extends EventObject {
    private String hora;

    public RelojEvent(Object source, String hora) {
        super(source);
        this.hora = hora;
    }

    public String getHora() {
        return hora;
    }
}
```

```
public interface RelojListener extends EventListener {
    void horaActualizada(RelojEvent event);
}
```

```
public class Reloj extends Thread {
   private int horas = 0;
   private int minutos = 0;
   private int segundos = 0;
   private ArrayList<RelojListener> oyentes =
                          new ArrayList<RelojListener>();
   public synchronized void addRelojListener(RelojListener oyente){
        oyentes.add(oyente);
   }
   // faltan get y set para minutos y horas
   public int getSegundos() {
        return segundos;
   }
   public void run () {
       while (true) {
           segundos++;
           try {
                TimeUnit.SECONDS.sleep(1);
           }catch (InterruptedException e) {
                e.printStackTrace();
           }
           setSegundos(segundos);
       }
  }
```

MDE: Ejemplo “actualizar varias ventanas”

```
//continua clase Reloj
public void setSegundos(int newSegundo) {
     // aquí código que actualiza los contadores de segundos, minutos y horas
      String horaActual = String.format(
                                   "%02d:%02d:%02d", horas, minutos, segundos);
      // Notificar a los oyentes cuando la hora cambia
      RelojEvent evento = new RelojEvent(this, horaActual);
      this.notificarCambioHora(evento);
}
private void notificarCambioHora(RelojEvent evento){
        for (RelojListener oyente : oyentes)
                oyente.horaActualizada(evento);
}
```

```
public static void main(String[] args) {
   Reloj reloj = new Reloj();
   VentanaReloj1 vr1 = new VentaReloj1();
   VentanaReloj2 vr2 = new VentaReloj2();
   reloj.addRelojListener(vr1);
   reloj.addRelojListener(vr2);
   reloj.start();
}
```

```
public class VentanaReloj1 extends JFrame implements RelojListener {
    private JLabel labelHoras;
    private JLabel labelMinutos;
    private JLabel labelSegundos;

    public VentanaReloj1() {
        super("Ventana Reloj Digital 1");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JPanel panelHora = new JPanel();
        panelHora.setLayout(new FlowLayout());
        getContentPane().add(panelHora,BorderLayout.CENTER);
        SimpleDateFormat sdf = new SimpleDateFormat("HH");
        labelHoras = crearLabel(sdf.format(new Date()));

        sdf.applyPattern("mm");
        labelMinutos = crearLabel( sdf.format(new Date()));

        sdf.applyPattern("ss");
        labelSegundos = crearLabel(sdf.format(new Date()));

        panelHora.add(Box.createRigidArea(new Dimension(0,150)));
        panelHora.add(labelHoras);
        panelHora.add(labelMinutos);
        panelHora.add(labelSegundos);
        panelHora.setAlignmentY(Component.CENTER_ALIGNMENT);

        pack();
        setVisible(true);
    }
```

```
public class VentanaReloj1 extends JFrame implements RelojListener {
  // continua

   private JLabel crearLabel(String valorInicial) {
        JLabel etiqueta = new JLabel(valorInicial);
        etiqueta.setFont(new Font("SansSerif", Font.BOLD, 32));
        etiqueta.setOpaque(true);
        etiqueta.setBackground(Color.BLACK);
        etiqueta.setForeground(Color.WHITE);
        Border borde = BorderFactory.createLineBorder(Color.BLACK, 2);
        etiqueta.setBorder(borde);
        return etiqueta;
    }

    @Override
     public void horaActualizada(RelojEvent event) {
          String hora = event.getHora();
          String[] partes = hora.split(":");
          labelHoras.setText(partes[0]);
          labelMinutos.setText(partes[1]);
          labelSegundos.setText(partes[2]);
     }
}
```